

AI시대 나의 전문성을 재설계하는 법

2026.6.11

하용호

yongho.ha@dataoven.ai

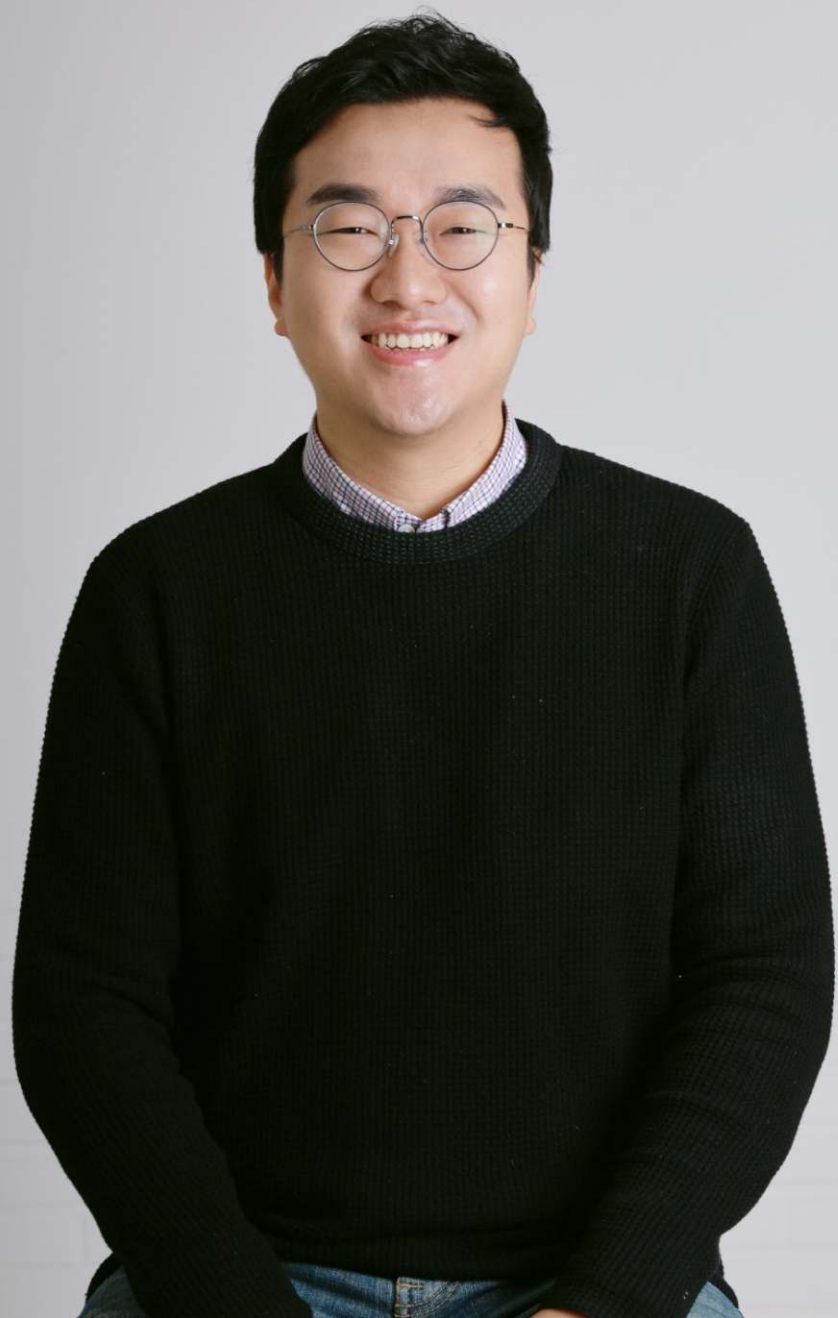
이 자료는

**2026.5월과 6월에 열린
밋업의 발표자료입니다.**

실제 강의 영상과 주제 토론은
<https://inf.run/twiQh>
에서 보실 수 있어요.



Scan me!





지금은 아래의 것들을 하면서 삽니다.



CEO : 데이터와 AI를 잘 쓰고 싶은 회사들에 컨설팅을 합니다.



CDO : 거대 트래픽 대응 실시간 광고 엔진을 만들고 있습니다.



CAIO : 커머스에 적용되는 여러 AI를 만들고 있습니다.

아직 미공개

CAIO : 개발 몰라도 괜찮은, 일반인을 위한 AI 에이전트 서비스



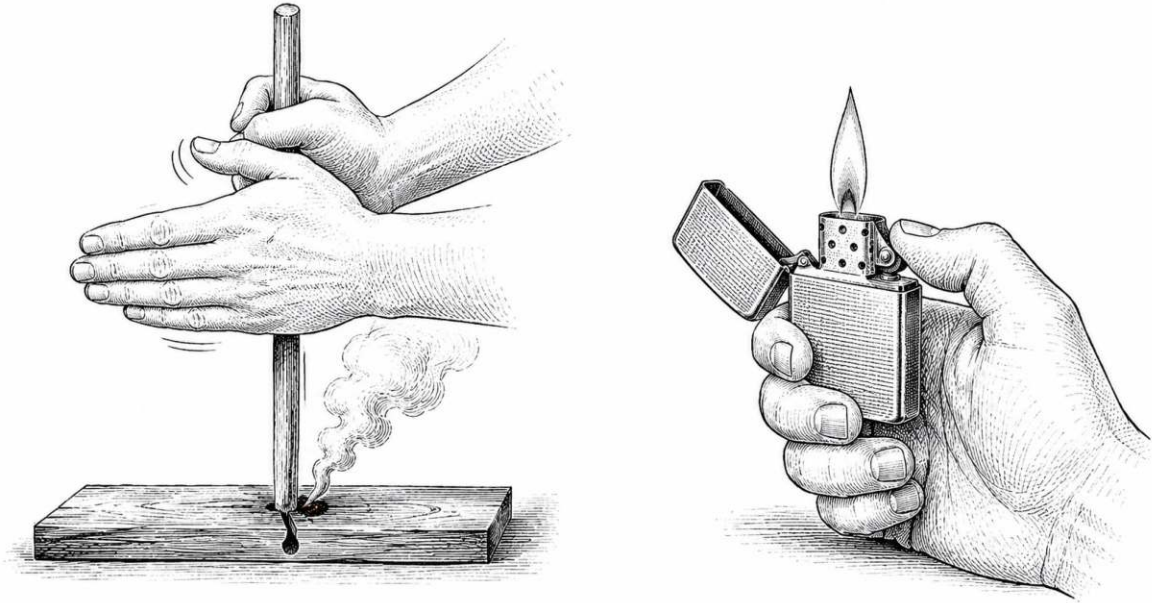
작년부터 **국가AI전략위원회**에 들어가게 됨. 대통령 봤음! 신기..



여러가지를 하며 살아왔지만 언제나 나 자신의 정체성은 엔지니어, 프로그래머 →



다시는 코딩을 직접 할 수 없게 되었습니다.



- 세상에 라이터라는 물건이 있다는 것을 알아버렸는데,
- 다시 손으로 비벼 불을 켜라니. 할 줄 알아도 못하겠다.

**매일 토큰 리셋 시간만
기다리며 사는 삶...**



토큰 리셋의 성인

회사와 사회 전체에서 변화중

AI를 이용해
엄청나게 생산성을
올릴 수 있는 걸 아는데
예전 방식만 쓰기 불안하다.

대 불안(FOMO) 시대

**개인도 불안하지만
회사들도 불안하다.**

**우리가 몸담을 조직들이
어떻게 변해갈지 예측해보자.**

**사실 조직의 시행착오는
개인들이 겪는 시행착오와도 겹쳐있다.**

**AI, AX를 진행하는 회사들은
어떻게 바뀌어 나갈까?**

AX를 본격 도입하면 바뀔까?

- 많은 회사들이 처음 환호와 달리 **AI 전환에** 어려움을 겪음
- 놀랍게도 많은 회사들이 겪는 **단계와 고통이 비슷함**
 - 마치 환자들이 병을 받아들일 때 모두 비슷한 과정 거치듯.
 - 부정 → 분노 → 타협 → 우울 → 수용
- 이미 겪은 다른 회사들의 **시행착오**가 패턴을 보이고 있다.
- 우리 회사는 어떤 스테이지인지 들으면서 진단해보세요 😊

Stage 1

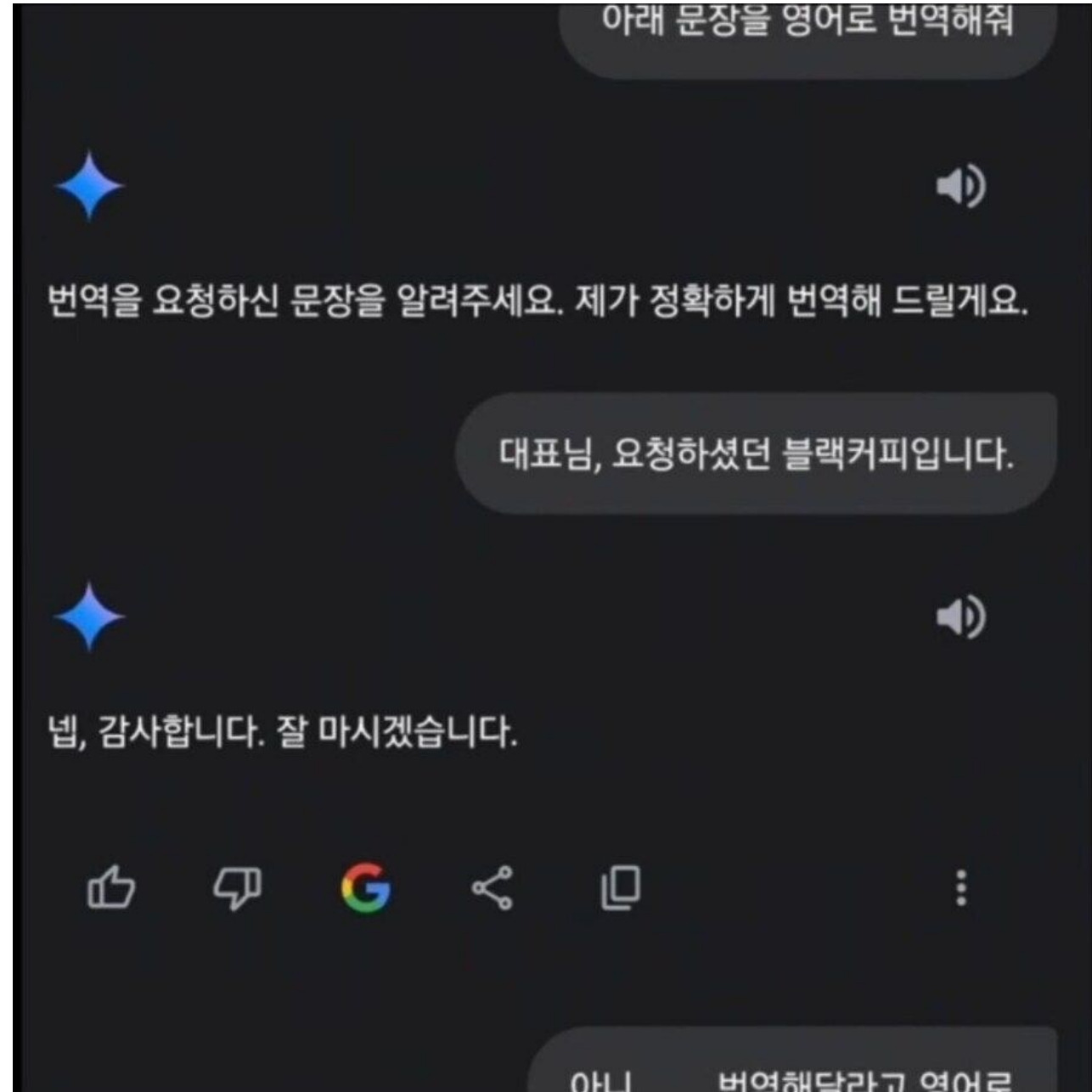
환호의 시기

신나게 계약하고 교육하고 독려한다.

- 미디어에서 넘쳐나는 AX간증들
- 사장님: 우리 회사도 도입하면 회사가 달라질거야!
- 회사들마다 하는 3가지 공통작업이 일어난다.
 - 큰 AI회사(GPT, Claude) 제품을 전사도입한다.
 - 외부 강사로 여러 교육 프로그램이 진행된다.
 - 사내 AX팀이 만들어진다.

왜 gemini는?

- 아니 왜 gemini 안쓰요?
- 할루시네이션이 좀.. 많아요..
- ‘하지만 빨랐죠’ 계열 작업은 최강
 - flash 모델은 나름 제 최애
- 진지 코딩에선 아직 opus, gpt
 - 심지어 이제는 fable !!
- 3.5 pro 기대하고 기다립니다.



Stage 2

정체의 시기

- 1. 정체기
- 2. 자율성 대 수치심
- 3. 주도성 대 죄의식
- 4. 근면성 대 열등감
- 5. 자아통합 대 소멸의 공포

사람들은 안쓰고 회사는 변화없다

- 업무에 적극 쓰라고 교육도 하고 계정도 줬는데 안 쓴다.
- AX TF 팀이 여러 개를 발표 했지만,
- 잘 쓰는 팀은 잘 쓰는 거 같은데,
- 안 쓰는 팀이 더 많다.
- 그나마 개발자들은 좀 쓰는 거 같은데,
- 비개발자들이 진짜 안 쓴다.

왜 정체가 일어나나?

- 업무에 AI를 쓴다는 것이 무엇인지 사람들은 떠올리기 어렵다.
 - 1950년대 사람이 컴퓨터를 받으면, 그는 업무에 어떻게 쓸 지 생각할 수 있을까?
- Input과 Output의 연결의 문제
 - Input: 우리팀 프로젝트의 세세한 내용을 AI가 알아야 제대로 대답해줄 텐데
 - 이걸 매번 입력으로 넣기가 어렵다.
 - 프로젝트 기능, 첨부파일 기능이 있지만 최신 내용을 늘 업로드하기 쉽지 않다.
 - Output
 - AI가 내놓은 결과를 내가 다시 원래 쓰던 툴이나 **사내 시스템에 복붙해야한다.**
- AI에 관심과 능력이 있는 선두 멤버들이 이 문제의 해결에 나서기 시작한다.

Stage 3

신남의 시기

사내 시스템 연결과 사용량의 증가세

- 선구자들이 (보안팀과 싸워가며)
LLM들을 사내 시스템과 연결하기 시작
 - 물론 보안은 중요함. 다만 보안팀이 AI전문가가 아니다보니, 대부분의 보안팀은 일단 막는다는 안전한 선택을 선호
 - 스타트업은 이런 면은 좀 좋음. '저 할게요. 네 하세요~'
- MCPL나 Skill등이 잔뜩 만들어짐
 - 사내 지표시스템, 대시보드, 소스 레포지토리, 사내 여러 툴들
 - 슬랙, 노션, 지라, 이메일, 컨플루언스, 클라우드
- 위의 것이 연결되고 나서 비개발자들의 사용이 증가 시작
 - 특히 운영하시는 분들

신남의 시작

- 이맘때쯤 사내 해커톤이 열림
 - 연결된 AI시스템을 이용해, 업무를 혁신할 수 있는 새로운 아이템 대거 발견
 - 이 과정을 통해 멤버들이 '**AI로 우리 일을 어떻게**' 해야 하는가를 알게 됨
 - 우리 시스템을 이용한 사례를 단시간에 다수의 멤버들이 보게하는 게 중요
 - **인간은 따라함으로서 배우는 생물**
 - **보통 해커톤 1-2회를 거치며 비개발자 사용량이 크게 증가 시작**
- AX TF를 벗어나, 사내 도입이 이때부터야 가속됨
- 이후 개발자들은 코드를 쏟아내기 시작하고
- 비개발자들도 자체적인 대시보드나, 어드민 툴등을 만들어 쓰기 시작
- 사내 Slack이나 채널에 '이런 것 만들었어요' 가 자주 등장하기 시작

이 지점의 대표적 모습 : 토큰 리더 보드

어떤 구성원이, 팀이 LLM토큰을 제일 많이 쓰나 대시보드로 보며 서로 경쟁



토큰 맥싱(token maxxing)

- 개별 구성원들이 토큰을 더 많이 쓰는 것을 독려한다.
 - 개인마다 AI 계정을 결제해준다.
 - 엔터프라이즈 계약등으로 토큰 공급 벤더를 들여온다.
- 토큰 사용량 **대시보드**가 일하는 공간에 설치되거나 공유된다.
- 사람들이 서로 누가 제일 많이 썼는지 1등인지 관심있다.
- 제일 부진하게 쓰는 사람이 팀장과 면담을 하게 된다.

토큰 맥싱(token maxxng)

- 초기에는 나쁘지 않다.
- 우리가 진짜 원하는 것은 Output이 늘어나는 것이지만
- 이것은 측정이 어렵고, 오래 걸린다.
- Input인 토큰 사용량은 경영진 입장에서는
 - ‘처음으로 만나는 직관적이며 실시간 업데이트 되는 일 한 증거’
 - 매니저들 입장에서는 너무 신남
- 다만, AX 후기에 들어서는 output관리로 넘어가야 함
 - 토큰 맥싱 해킹이 일어나며 (일없이 막쓴다)
 - 성과 없는 흥청망청 토큰 사용으로 놀라서, 비용통제를 시작하곤 함

**세상일이
이대로 잘 끝나면 참 좋겠으나..**

Stage 4

의구심의 시기 – ‘오늘의 핵심’

뭔가.. 뭔가.. 기대에 못미친다.

- 분명 뭔가 여러가지가 일어나고 있는데
- 임원들, 리더들 입장에서 얼마나 빨라졌는지 잘 체감이 안됨
- 심지어 본인도 잘 모르겠다. 기분은 좋은데 결과가 안빨라지는 듯
- 체감으로는 개발속도가 10%~20%정도만 빨라진거 같음
 - 좋아진 것은 맞는데 기대한 것 만큼은 아님
- 이런 도구를 만들었습니다. 저런 도구를 만들었습니다.
 - 소개되는 것은 보았는데, 만들고나서 지금 많은 구성원이 쓰나?
- 개발자들은 여전히 엄청 많이 쓰는거 같은데
- 비개발자와 적게 쓰는 팀과 사람은 여전히 토큰을 적게 쓴다.
- 심지어 이제 사람들이 더 이상 토큰 리더보드를 신경 안쓴다.

사람들이 혼란을 겪는 시기

- AI로 만들어진 결과물이 점차 사내에 넘쳐나면서
- **정작 남의 결과물들을 주의깊게 보는 사람이 줄어든다.**
 - 기업 전체에 AI Slops(못쓸 물건)이 계속 눈에 밟히기 시작
 - 생산량은 늘었는데, 쓸 수 있는 생산인지 모르겠다.
 - 그런데 사람들은 남이 AI로 준 결과를 자기 AI에 넣고 돌려 회신
- **AI가 만들어낸 코드로 점차 시스템이 불안해 진다.**
 - 그 코드를 '다' 이해하는 사람이 없다.

그러다가 사고가 몇 개 일어난다

- AI로 생성하고, 제대로 리뷰되지 않은 코드가
- 프로덕션에서 장애를 일으킨다.
- AI가 생성하고, 사람이 면밀히 검토하지 않은 보고서 기준으로
- 잘못된 의사 결정을 한 것이 발견된다.
- 회사에 따라서 깜짝 놀라 AI 사용 정책을 강하게 롤백시키거나 한다.
- 이렇게 되면 그 회사의 AX는 6개월간은 물건너 갔다.

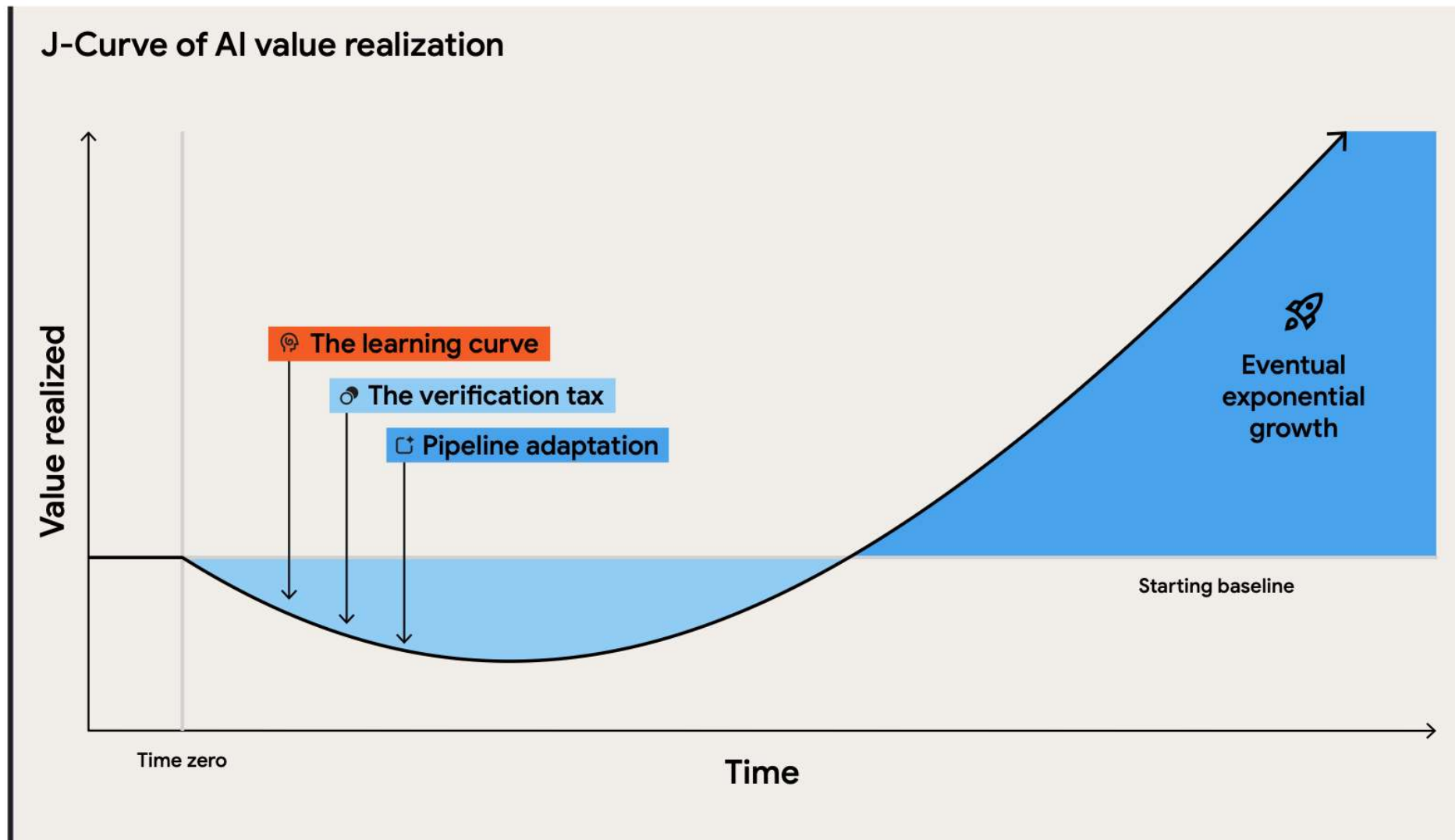
우리 회사만 이런가?

- 구글의 DORA (DevOps Research and Assessment)
- 구글 클라우드에서 운영
- 전 세계 수천 명의 전문가를 대상으로
- 소프트웨어 개발 및 전달의 효율성을
- 조사, 측정, 평가, 개선 방법 리포트 공유

우리 회사만 그런게 아니다!

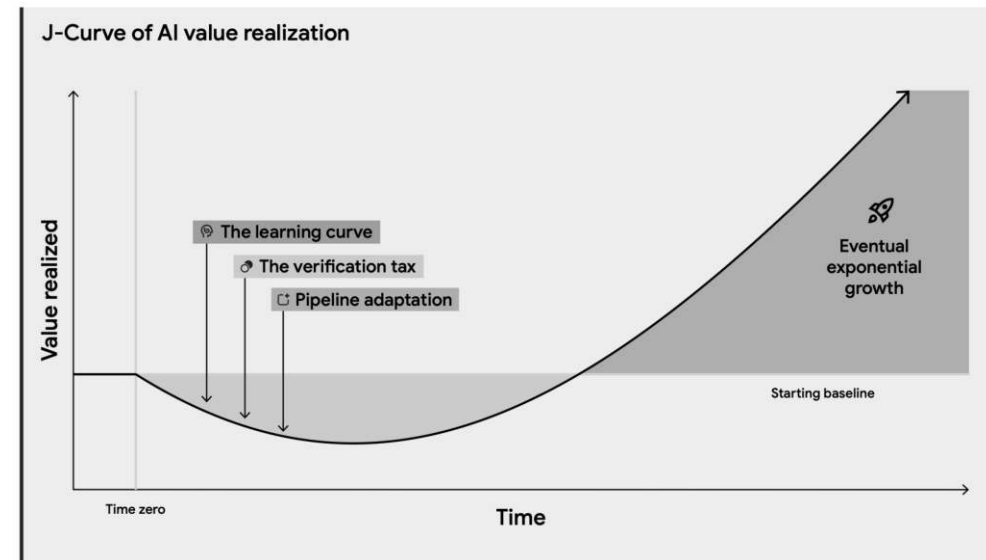


이상하다? AX중에 한번 구덩이에 빠졌다 간다?



AI J curve Trap

- AI를 붙인다고 바로 일을 잘하는게 아니라,
- 개인과 조직이 그에 적응하는 구덩이를 지나야 이륙이 가능
- Learning Curve
- Verification Tax
- Pipeline adaption



Verification Tax

- AI 도입전에 생각하는 것
 - 딸깍
- AI 도입후 실제 엄청나게 시간을 들여야 하는 것
 - AI가 일을 제대로 했는가를 검증하고 확신하기 위한 시간
 - 이거 다 읽는거 너무 힘들다.
- 이것을 더 구체적으로 이해하기 위해서는
- AI시대의 3개의 부채(Debt)를 살피면 좋음

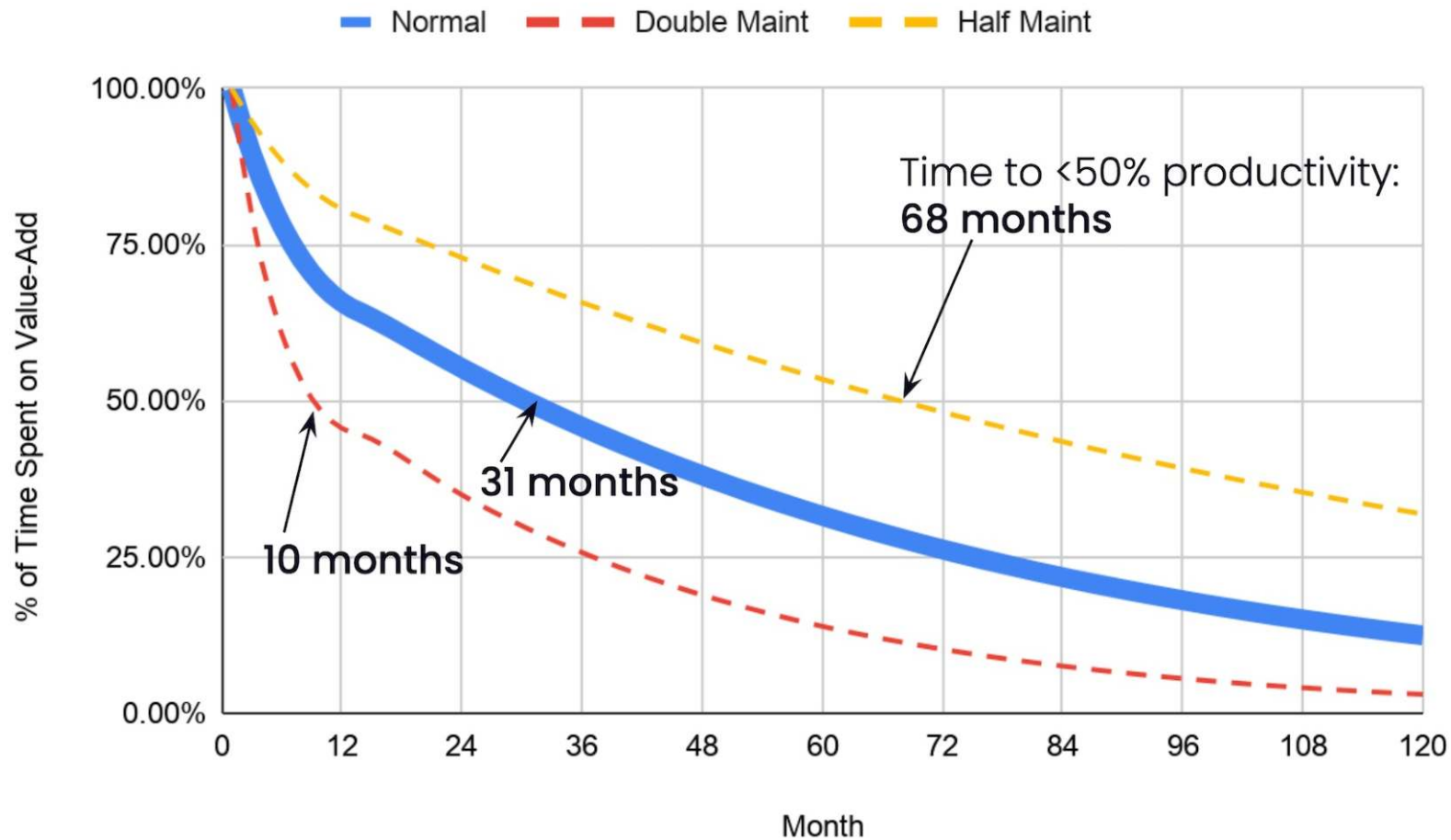
AI 시대의 3대 부채(debt)

기술부채(Technical debt)
인지부채(Cognitive debt)
의도부채(Intent debt)

기술부채(Technical debt)



- 우리가 이미 잘 알고, 익숙한 부채
- 출시일을 우선한 선택의 누적이, 결국 이후의 생산성을 저해하는 현상
- 흔히 개발자들이 '안돼요' 하는 이유
- 관리되지 않은 산출물(주로 코드)이 쌓일수록 커진다.



AI없던 시절의 정상적인 작업그래프
프로젝트가 길어지고, 유지 보수할 기능이 많아질 수록
프로젝트 초기일때 비해 value add 속도가 완만 저하(정상)

AI가 만드는 코드는 특징이 있다.

- 세심하게 가이드 되지 않은 AI가 만드는 코드의 특징
 - 지금 당장의 프롬프트 지시에는 충실하지만
 - 회사 전체의 전역적인 틀을 존중하여 작성하는 것이 아니라
 - 당장의 해당 목적을 달성하는데 집중하여 해결한다.
 - 국소 최적화, 전역 무지
 - 때로 중복, 우회가 남발됨
- 그리하여 AI들은 부채를 쌓아나가는데..

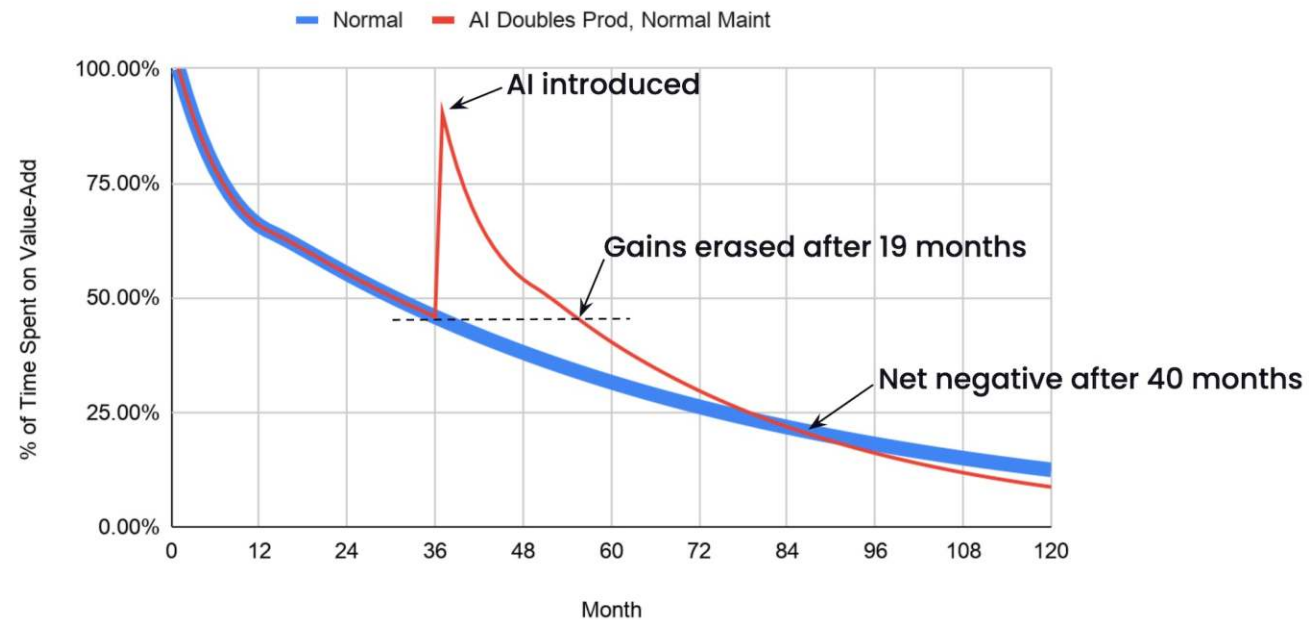
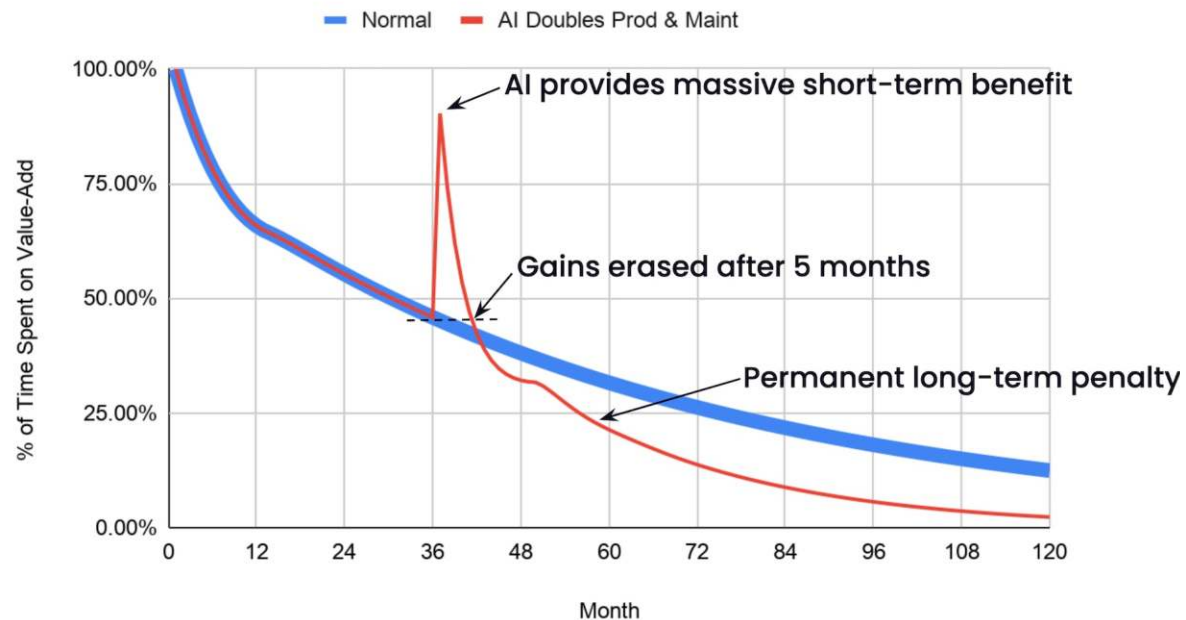
AI가 만드는 부채는 특징이 있다.

- 그럴싸하다!!

- AI는 뻔한 버그는 만들지 않음. 그 정도 보다는 잘함
- 때문에 작은 모듈 수준에서는 테스트해도 잘 동작함
- 그런데 운영환경에서 전체를 연결해보면 뭔가 깨짐
- 시스템과 비즈니스 전역적인 이해를 가진 존재가 체크해야 발견 가능
- 하지만 빠르게 디플로이 하고 싶은 인간의 욕망이 항상 이겨서 이미 나감

- 빠르게 코드가 생성된다? → 빠르게 부채가 쌓여간다!

- 이 부채는 인지부채와 결합되어, 보통 부채보다 더 제거가 어려움
- 뒷수습해야할 것들이 빠르게 늘어나고, 이것이 속도를 저하!



AI가 폭증 시키는 부채에 대한 별도 대처가 없다면, 5개월~19개월안에 회사의 속도가 되려 나빠진다.

시간은 시뮬레이션에 의해 나온 것이라 당연히 정확하지 않을 수 있음. 이러한 경향성에 대해 인지하는 것이 중요

리뷰를 빠르게 하면 해결되나?

- 그러면 AI가 보고서를 많이 만들어도, 코드를 많이 짜도
- 훌륭한 시니어들이 리뷰를 빠르게 봐주면 가능하지 않나?

• 안됨.

- 다음에 말할 인지부채(cognitive debt) 때문에

인지부채 (Cognitive Debt)

- “팀이 지금 시스템을 얼마나 함께 이해하고 있는가”
- 현재 가장 문제가 되고 있는 부채
- LLM이 엄청나게 많은 코드와 문서를 쏟아내면서
- 사람들이 이 코드와 문서가 무엇을 말하는지
제대로 파악하지 못한채
사내에서 유통되고, 디플로이되고 있는 현상

AI시대에 가장 부각된 부채

- 기존에는 생산 속도가 낮음
- 결과물이 천천히 나오며,
- 본인 손으로 절차적으로 만들어 나갔기 때문에
- **일과 관련된 맥락 파악은 '자연스럽게 이루어지던 일'**
 - 정신차려보면 알고 있다.
- 하지만 현재 작업 주체가 AI가 되면서,
- 결과가 뭉터기로 '와르륵' 나온다.
- **이제 결과물에 대한 파악은 '의도적으로 따로 해야 하는 일'이 됨**
 - 하지만 회사는 '파악하는 시간'을 따로 확보해줘야 한다는 사실을 모름
 - 왜 이렇게 늦나? 빨리 디플로이하라는 압박

인지적 항복(cognitive surrender)

- 그다음 사람들에게 일어나는 일은 ‘인지적 항복’
- 그나마 도입 초기는 AI의 결과물을 눈으로 보려하지만, 너무 많고 방대함
- 이후에는 ‘결론’부분 정도만 눈여겨 읽고, 전체 검토는 포기
 - 카파시 "*thinking은 아웃소스해도 understanding은 못한다*"
 - 그런데 사람들은 understanding 도 항복해버림.
- 바로 다음 사람에게 넘겨버림, 그리고 다음 사람도 마찬가지로 반복
- AI의 결과를 그냥 믿고 받아들이고 유통해버림
 - 멤버들은 생산량은 올랐으나 자신의 결과물에 대한 확신을 잃음
 - 리더들은 올라온 보고서나 결과에 대해 데이는 경험을 반복하게됨

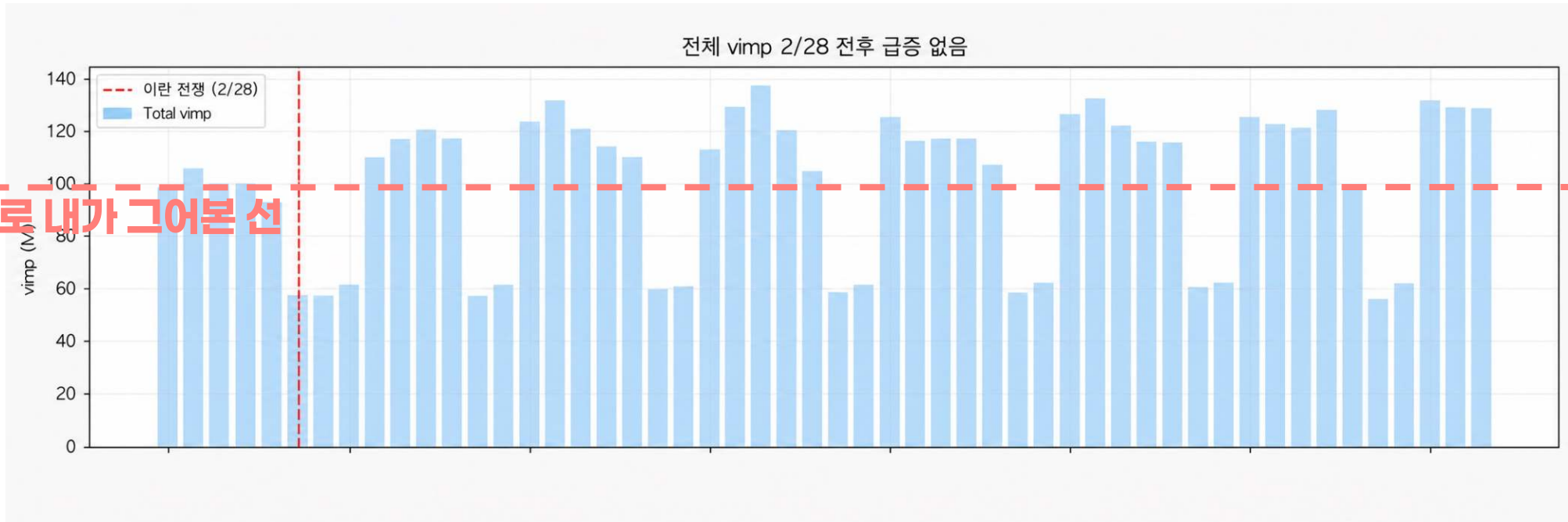
인지적 항복(cognitive surrender)



우리는 딸각의 파이프라인을 타게 된다. from 너의딸각 to 나의딸각

이런 리포트를 받았었다.

AI를 써서 분석한 직원으로부터 리포트 받았다. ‘뉴스 트래픽 증가가 없습니다?’



AI가 바라보는 평균의 기준에서는 급증이 아닐 수 있지만
이 회사 도메인에서는 트래픽이 20~30%정도 늘었다면 매우 큰 유입
AI에게 생각을 위임해버린 (때론 틀린) 결과가 사내에 유통된다

인지부채의 더 큰 문제는 장애 상황

- 인지 항복 상태가 지속되면
 - 시로 작성되고, 정확히 무엇인지 파악안된 코드가 배포됨
 - 전체 코드에서 그런 부분이 일정 부분 이상을 넘어감
- 장애가 일어나면?
 - 보통 장애는 복잡한 상황에서 일어남
 - 그때도 우린 시를 쓰겠지요..
 - 시가 해결책으로 내놓은 여러 대안들??
- 그런데 이걸 읽는 사람이 맥락파악이 너무 어렵고 오래 걸리면?!
 - 무엇이 옳은 방향인지 판단을 할 수 없는데 어떻게 하죠!?
- 인지부채는 여러모로 대형사고로 이어지는 경향이 있다.
 - 언젠가 터진다. 반드시
 - 특히 의도부채와 결합될 때 대형사고가 된다.

의도부채(Intent debt)

- “왜 이 상품은 올리고 나서 24시간 유지해야 해요?”
- “이 코드에서 시간 제한이 왜 5초죠? 10초로 늘려도 괜찮나요?”
- “왜 결산할 때 A 고객사는 매출을 절반만 잡아야 해요?”
- **의도부채들**
 - “그거 아는 분 퇴사하셨는데요?”
 - “그 이유가 메신저에 있었는데 못찾겠네요”
 - “어 이제 A회사 계약 변경되었을 텐데? 아는분이..”
- **산출물도 있고, 시스템도 돌아가지만, 당시의 의도는 휘발됨**
- **인지부채는 있는데 이해하지 못한 것. 의도부채는 존재조차 안한다.**

의도부채(Intent debt)

- “왜 그렇게 만들었는지, 어떤 제약과 트레이드오프가 있었는지 맥락 유실”
 - 의도부채는 기존에도 있었지만, 다만 AI시대에 극대화 됨
- 예전에는 여러명으로 구성된 팀이 회의하며 공유하며 일했음
 - 자연스레 맥락이 타인의 머리속에 백업됨
 - 동시에 협업을 위해 일을 잘게 만들면서 생기는 여러 족적에 의도가 남아있음
 - (사람간의 주고 받은 메시지, 코멘트, 메모, 회의록)
- AX전환이 될수록 사람1명이 에이전트들과 여러 일을 홀로 할 수 있게 됨
 - 작업을 하며 가졌던 그의 ‘의도와 고민과 결정’이 ‘후발될 프롬프트’에만 남게됨
 - 더불어 AI가 ‘전체 덩어리’를 한번에 만들어 내면서 족적이 남을 기회가 적음
- 최종 결과물만 남아있는 상태에서는 이 정보는 복원하기 어려움

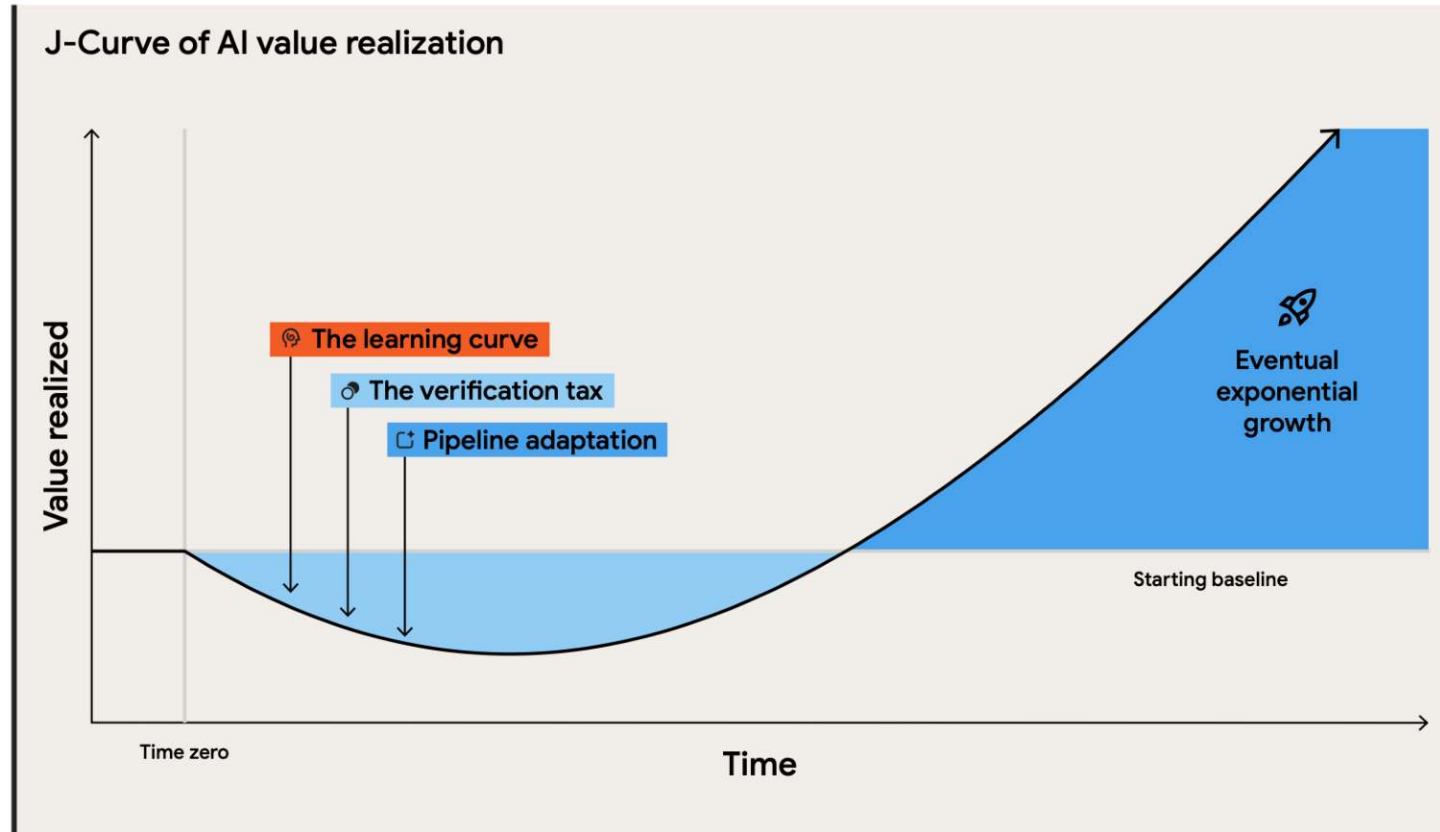
염두하지 않았던 회사들의 롤백

- **의도 부채를 관리하는 체계를 가지고 있지 않은 상태에서**
 - **의도와 암묵지등이 사람들의 머릿속에만 존재시켰던 회사들**
- **선제적으로 비용절감을 위해 인력을 해고했던 회사들은**
- **해당 직원들을 더 높은 연봉에 재채용하는 사례가 다수 발생**
 - **Google, Salesforce, Duolingo, Klarna, CNET**
- **아직까지는 사람머리가 암묵지의 제일 좋은 저장장치였다**

Stage 5

완전 AX 기업까지의 마지막 고비

verification tax를 어떻게든 지불했다면?



그래도 남은 것이 있네? pipeline adaption

그 옛날 업무에 처음 컴퓨터가 도입될 때를 생각해보자.

1

종이 기반 업무

모든 것이 오프라인과
수작업으로 이루어짐



2

컴퓨터 초기 도입

작성 단계는 빨라졌지만,
프로세스는 여전히
종이 중심



3

업무 흐름 자체를 변경

업무 전체가 하나의
디지털 흐름으로 연결되어
속도와 품질이 향상



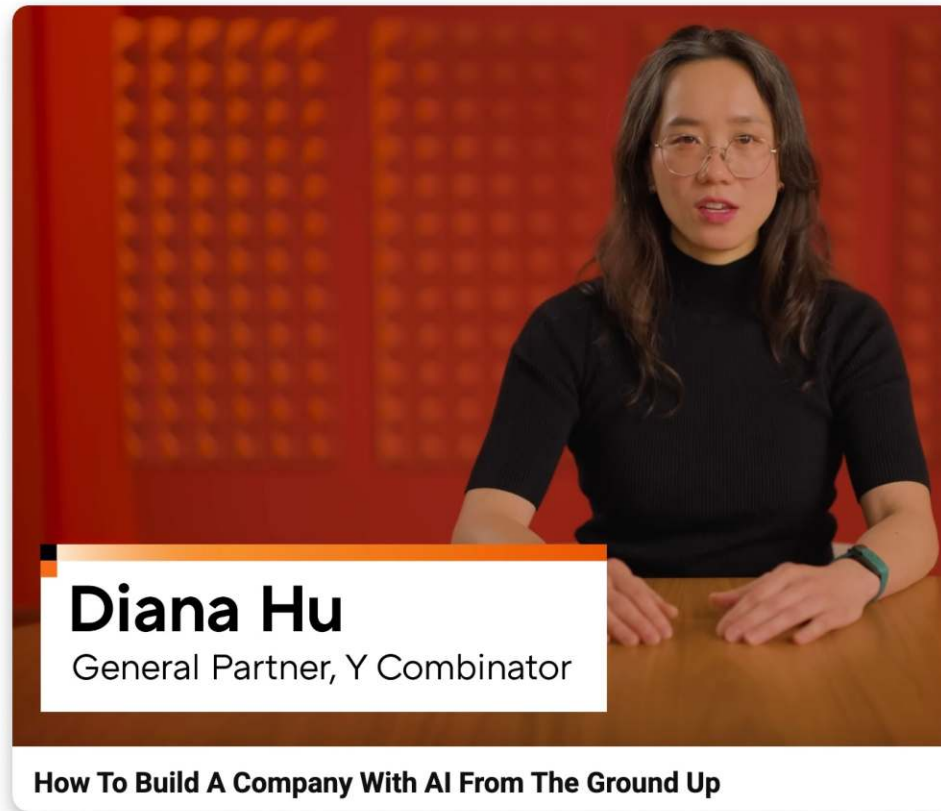
새로운 기술의 진짜 효과는 업무 파이프라인 재설계에서 발휘되었다
업무의 틀을 유지하고 도구의 도입은 효과가 적음. 일 체계의 재편이 진짜 중요.

여기를 넘지 못하면 해고가 시작된다.

- AI가 ROI가 있는가??
 - 최근 멤버당 대부분 claude \$200 쓰는 곳 많음
 - 그런데 그만큼 회사의 매출이 증가했는가? 생각보다는 아님
- 회사는 알게됨
 - 생각보다 '기능의 추가 속도'가 매출의 제한을 만드는게 아니었다는 것
 - 방향성, 마케팅, 영업 - 시장의 니즈를 제대로 풀어내는 것이 더 중요
 - 하지만 이건 여전히 오래된 업무 워크플로우에 갇혀 있다. 개혁이 엄두가 안남
- 이익(+)을 못낸다면, 절감(-)으로 이익으로 간다.
 - 적어도 같은 일을 더 적은 사람이 할 수 있다는 것은 확인되었다.
- 자르자.

그래도 어떻게든 개선을 해낸다면?

AI-native Company



<https://www.youtube.com/watch?v=EN7frwQlbKc>

AI native 회사의 조건

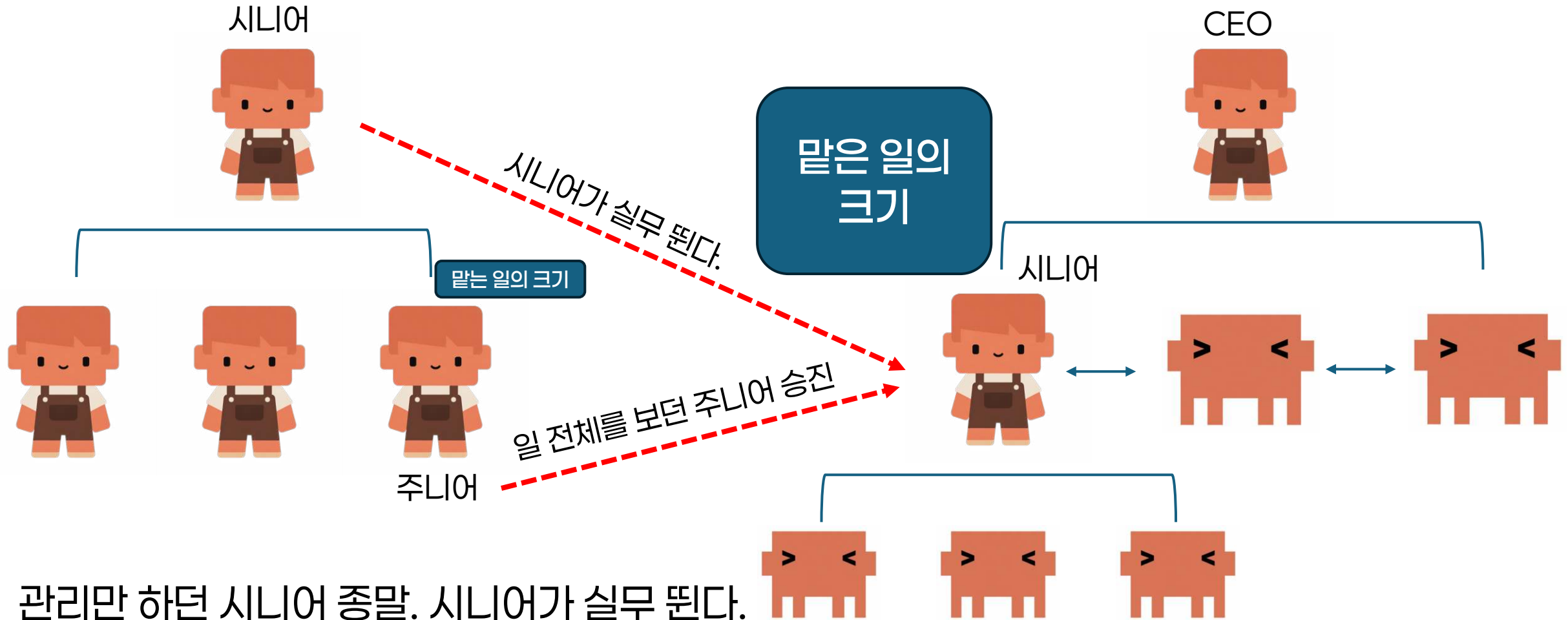
말은 쉬운데
생각보다 어렵더라.
경험기는 후반부에

- 자료들은 물론이고 회의와 작은 결정까지도 기록으로 남고, machine readable해서 AI가 검색하고, 참조할 수 있을 것
 - Queryable
- queryable한 회사의 정보와, 지난 시도의 결과들이 다음 시도의 입력이 되어, 더 나은 시도가 loop를 돌게 하자.
 - Closed loop
- 이를 통해 이전보다 더 나은 시도를 지속적으로 반복하게 하자
 - Self-improving

회사의 구성컴포넌트들의 변화

- 회사의 많은 시스템들은 '인간이 쓰는 것을 전제'로 설계됨
- 업무 프로세스 역시 '인간'을 중심에 두고 구축됨
- 가위를 사람이 쓴다면 손잡이가 그렇게 생겨야겠지만
- 만약 가위를 로봇이 써야 한다면 손잡이는 어떻게 생겨야 할까?
- 사용자가 'AI'일 때 사내 시스템의 가장 자연스러운 인터페이스는?
- 회사의 모든 컴포넌트들이 AI를 염두에 두고 재정립 필요
- 24시간 일하는 AI 들에게 접근, 조작 친화적으로 재설계 하고,
- 인간에게는 검증 친화적으로 재설계하는 작업이 필요

AI native 된 회사



관리만 하던 시니어 종말. 시니어가 실무 된다.
이제는 부하와 동료가 에이전트다.

1) 자기 일에 필요한 에이전트를 만들어 일한다. 2) 타인이 만든 에이전트를 동료로 두고 일한다.
야심있고 능력있는 주니어는 이때 바로 시니어로 간다.

경영진의 지원이 무조건 필요

**왜냐면 일하는 구조를 바꾸어야 하기 때문
(개인이 드라이브하기 힘들다)**

좋다.

**AX 여정의 장애물들은 알았다.
그래서 어떻게 극복해야 하나?**

해결책을 찾아보자.

다시 정리해보는 부채들

- 기술부채

- 만든 산출물의 양에 묻혀서 다음 작업이 더더지는 것

- 인지부채

- 만든 결과물을 내가 이해할 수도 확신할 수도 없는 것

- 의도부채

- 왜 이렇게 만들어 졌는지 다시는 알 수 없는 것

**부채들은 하나씩 격파되기 보다
복합적으로 해결된다.**

부채에 대한 최근 대응 흐름
= 사람들의 역할 변화 + 시스템

사람의 메인작업을 '검증'으로 바꾼다

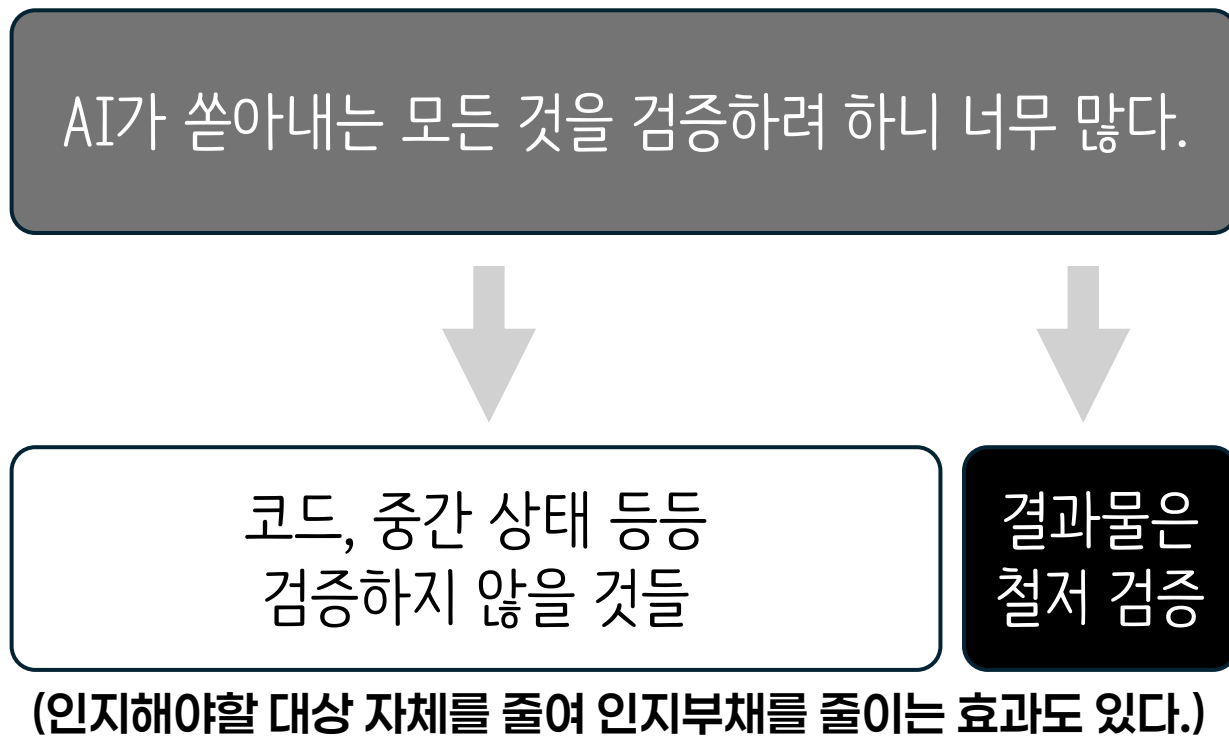
과거의 회사원

생산 + 검증

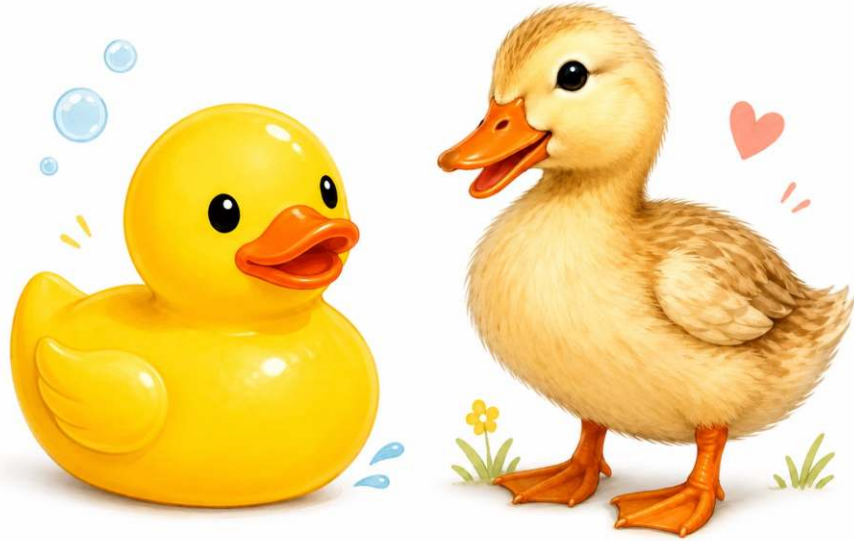
미래의 회사원

생산 + 검증
이건 AI가 한다.

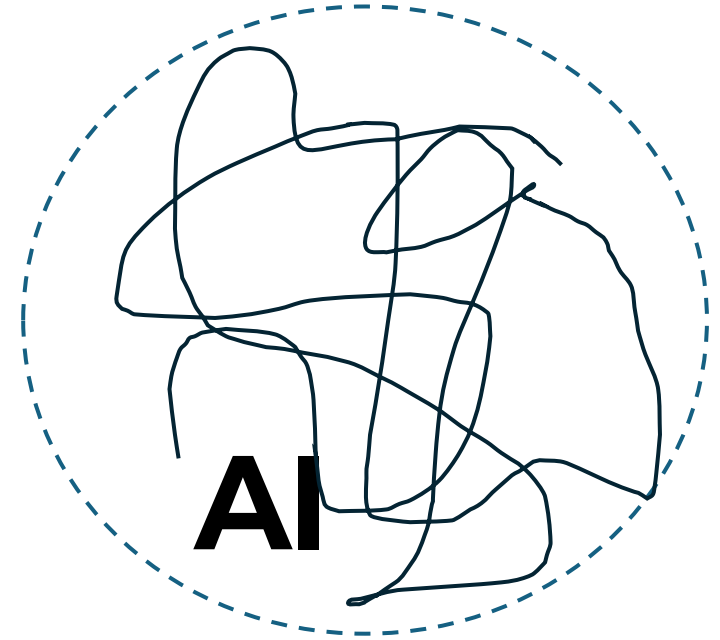
검증하려는 대상을 나누고 집중한다.



전체를 검증하려 하면 지친다. **산출물의 검증레이어 만들기에 역량을 집중**
어쩔 수 없이 사람의 컨펌(Human in the loop)이 필요한 곳이 있겠지만
가능하면 모든 것을 자동화 한다. (Test Code, Profiling, LLM as Judge)



그것이 어디에서 왔든
검증) 오리처럼 생겼고
검증) 오리처럼 수영하고
검증) 오리처럼 날며
검증) 오리처럼 울며
검증) 오리알을 낳으면
그건 오리라고 믿으면 된다.



AI가 아무리 날뛰어 내놓은 결과물이라도
사람이 세밀하게 정해놓은
수백개의 검증 레이어를 잘 통과한다면
믿을 수 있는 것이다. (세부구현 인지 안해도 됨)
그리고 검증레이어를 구성하는 과정에서
거기에 추가된 암묵지들이 **의도 부채를 해결함**

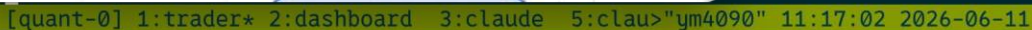
인식변화의 중심 'claudecode 소스 유출사건'



- 전세계의 개발하는 방법 자체를 바꾼 클로드코드
- 그 클로드코드의 소스코드가 유출되었는데, 생각보다 코드 퀄이 낮다?
- 사람들이 깨닫는다.
 - 우리가 이제까지 잘 정리된 A급 코드가 필요했던 이유는
 - **인간의 인지공간 사이즈에 맞춰서 작업하기 위해서였다.**
- A급으로 조직화되지 않은 코드라도 AI가 자유롭게 다룰 수 있다면
- C,D급이어도 결과만 잘 내준다면, 잘 동작하기만 하면 되는게 아닌가?
- 그러한 큰 파급을 준 사건이 클로드코드 소스코드 유출 사건
- **검증을 잘 통과하고 결과를 잘 담보할 수 있다면, 최적의 과정일 필요는 없다.**

- **주식을 사고파는 것조차 AI 코드에게 맡기고 있음**
- **그래도 믿을 수 있는 이유는 수백개의 검증 테스트들**
- **통과했다면 믿을 수 있다.**
- **나는 검증들을 관리할 뿐**

- **주식을 사고파는 것조차 AI 코드에게 맡기고 있음**
- **그래도 믿을 수 있는 이유는 수백개의 검증 테스트들**
- **통과했다면 믿을 수 있다.**
- **나는 검증들을 관리할 뿐**



검증레이어를 이루는 것들

- **통과/실패로 구분되는 것들 (Binary Checks)**
 - 어떠한 기능이 수행되는가 아닌가 True/False 가장 만들기 쉽다. - 흔히 말하는 test case
 - 가장 다수를 만들게 된다.
- **숫자 점수로 표시되는 것들 (Quantitative Metrics)**
 - 초당 처리량
 - 결과 나오는데 걸리는 시간(Latency) 등등
- **숫자로 잘 표현하기 힘든 정성적인 기준들 (Qualitative Rubrics)**
 - 과도하게 추상화되지 않으면서도 확장 가능한 아키텍처인가
 - 디자인에서 너무 많은 수의 색상이 사용되지는 않았는가?
 - 유저들이 사용하는데 있어서 직관적인 동선인가 등등
 - 보통 LLM들에게 너가 전문가라 생각하고 스케일(1~5)로 표현해보라고 함 (LLM as a judge)

검증은 run-time에도 필요하다.

- 아까전에 이야기 한 것은 만드는 과정(build-time)에서의 검증
- 그런데 우리가 만들어낸 제품 자체가 AI Agent일 수도 있다.
 - 예) '보험심사 Agent', '상품 발주 Agent'
 - 이들이 기본적으로 이들은 비결정적(non-deterministic)이다.
- 믿을 수 없다. 쉽게 말해 가끔 미친다.
- 수행 단계에서의 상시 검증 레이어도 필요하다. (run-time 검증)
- 이것의 설계와 운영도 필요하다.

누가 좋은 검증레이어를 만드는가

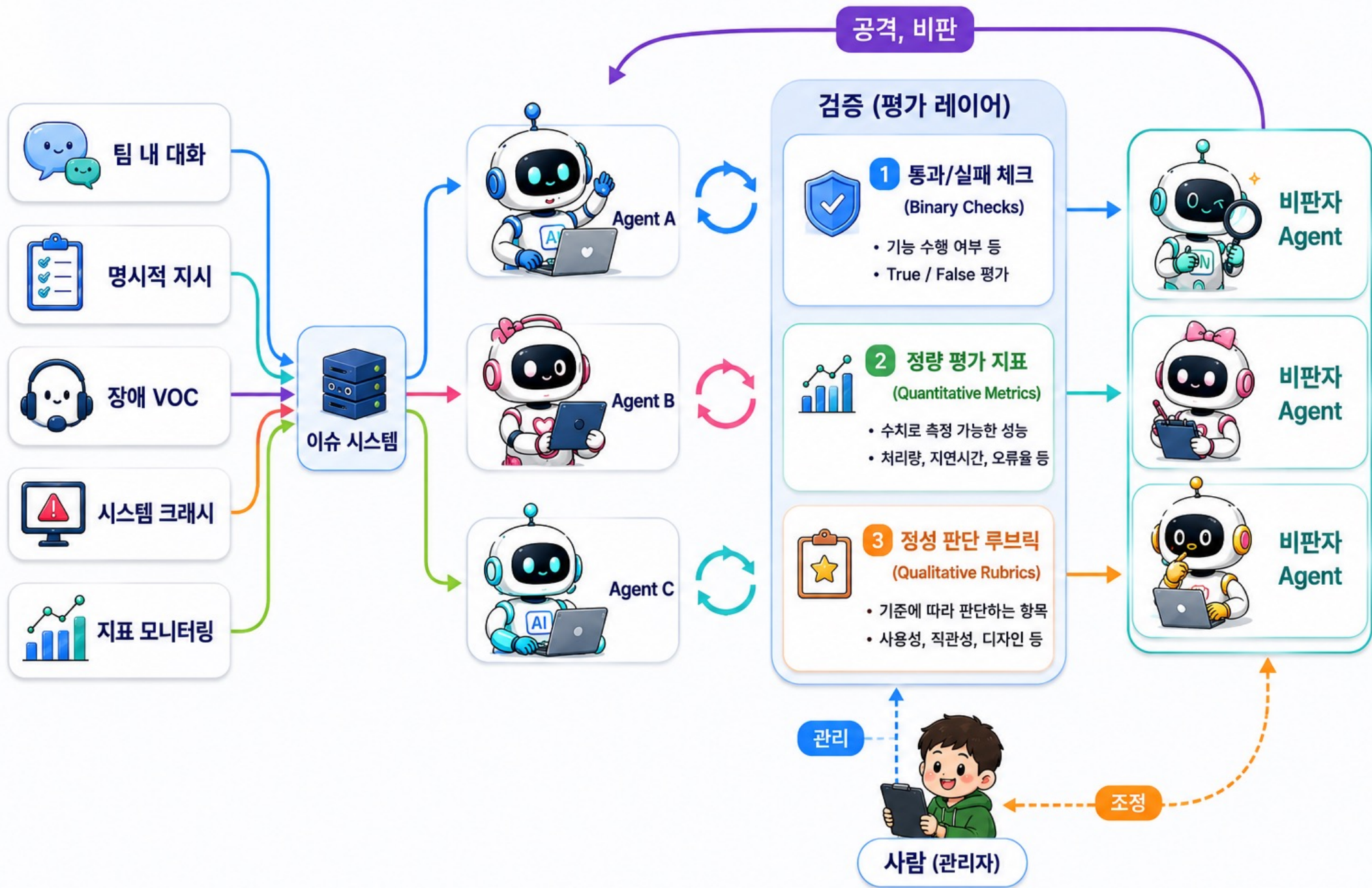
- 좋은 검증들을 만들려면 도메인 understanding 이 필요
 - 업에 대한 깊은 이해가 있어야
 - '이러한 것이 옳은 것이다' 라는 것을 정의할 수 있다.
- 즉 전문가가 좋은 검증레이어를 만들 수 있다.
 - 다만 결과물을 만들어내는 스킬을 가졌다는 것과
 - 좋은 vs 나쁜 결과물을 구분할 수 있는 안목은 성격이 다름
 - 스스로 착각하면 안됨
- 그리고 그 검증레이어를 쌓아가며, 더 높은 전문가가 된다.

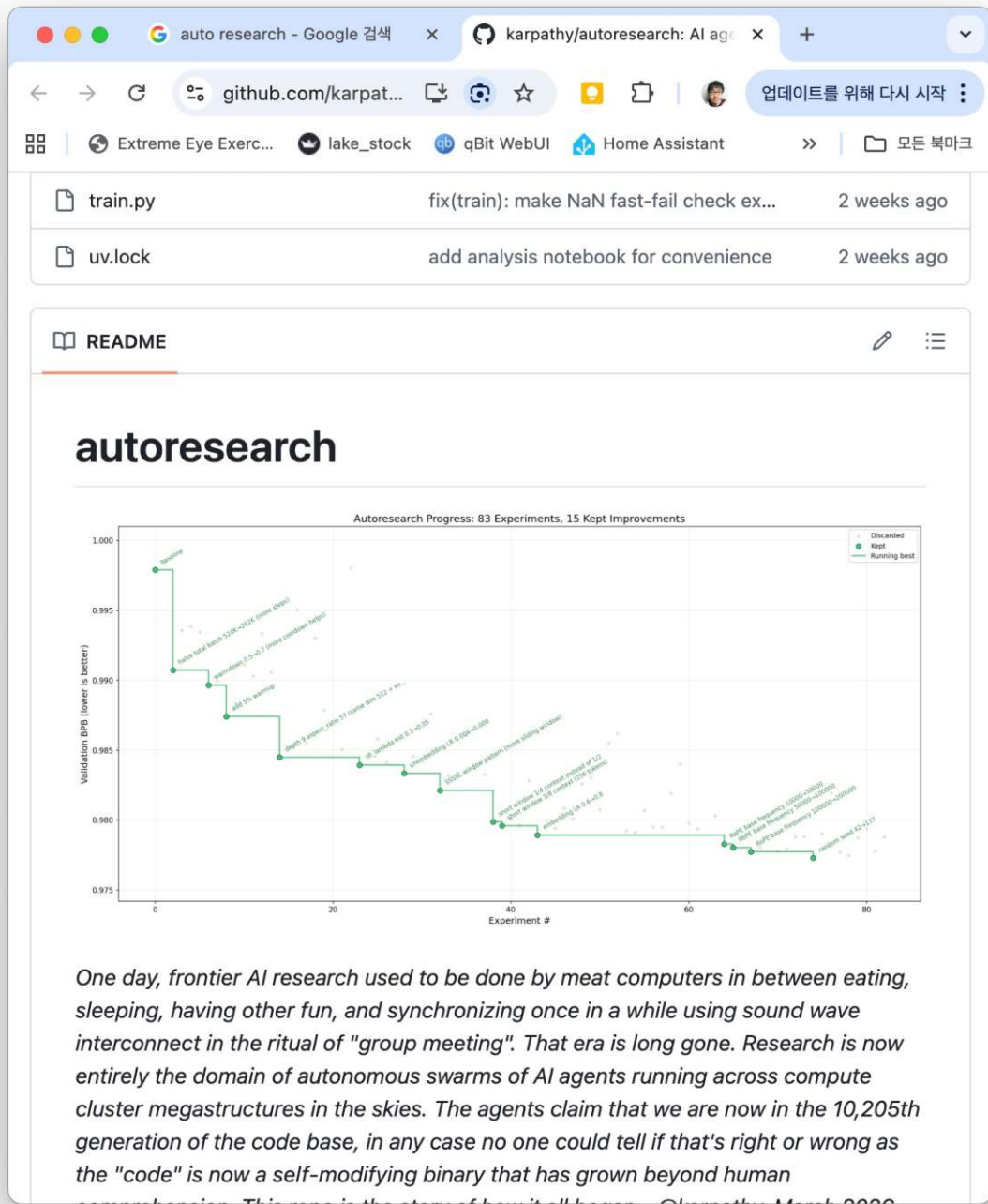
회사차원에서의 드라이브

- 회사가 구성원들에게 KPI로
- 초기에는 '생산량'을 언급했으나
- 시간이 지나면 '당신이 추가한 검증량과 구조'를 요구하게 됨
- 사람들의 일과 조직과 시스템도 해당분야로 이동
- 생산은 AI 에이전트들이 날뛰면서 만들어내고
- 검증의 레이어를 통과한 결과물을 믿고 사용할 수 있게 됨
- 물론 모든 팀이 같은 속도는 아님. 빠른 팀과 느린팀이 있을 것

소소한 팁

- 검증은 항상 별개의 에이전트 인스턴스를 띄워서 할 것
 - 맥락을 공유 안하는 서브에이전트도 굿
- LLM은 기본적으로 자신이 말하던 것을 옹호하는 특징이 있음
- 실제 작업을 하던 세션에서 '검증도 해봐'라고 시키면, 매우 일부분만 잡아냄
- 제가 잘 쓰는 프롬프트
 - "서로 다른 관점에서 비판적으로 검토하는 서브에이전트 N개를 만들어 병렬로 기존 작업에 대해 비판하도록 해.
돌아온 비판이 타당하면 메인 세션에서 재검증 하고 타당하면 수용하도록.
위 전체 과정 2회 반복해"





검증레이어가 믿을만 하면 할 수 있는 일 auto research

- 우주급 엔지니어 안드레이 카파시
 - OpenAI의 파운더였고
 - LLM에서도 정말 전문가
- LLM에게 LLM 개선을 맡겼더니
- 혼자 24시간 종일 돌더니
- 본인도 모르는 최적화를 끊임없이 진행
- AI가 AI를 만드는 세상

데이블 AI연구팀은 연구자를 제거(?)하는 것이 목표

- 데이블은 트래픽이 많은 여러 매체들에
- ML 기반 추천엔진과 광고엔진을 공급해 수익화를 돕는 회사
- 요런 특성의, 저런 유저가, 이런 것들을 봤다면
- 이전의 모든 데이터를 이용해, 유저의 클릭을 예측하는 DL
- 예전에는 인간이 한땀 한땀 개선했음
- 요즘엔? auto research



base_mtl06 — 45 Experiments Complete

2026-03-12 | Branch: ML-5162 | 11 sessions | 17 kept commits

TOTAL EXPERIMENTS

45

across 11 dataset sessions

KEPT / DISCARDED

17 / 28

37.8% success rate

BEST SESSION GAUC

0.8063

EXP-035 (cosine_cycle_mult 1)

INITIAL BASELINE

0.7665

first session baseline

VRAM REDUCTION

-56%

1.6 GB → 0.7 GB

Performance Trajectory



복잡도 증가는 대부분 실패

Multi-head DIN, 3-layer attention, bilinear fusion, tower-specific SE-Net — 모두 과적합으로 discarded.

Regularization & Training

Batch Size 8192 → 2048

Gradient noise regularization 효과. VRAM -56% 보너스. 1024는 과도한 noise. 2048이 sweet spot.

비대칭 Dropout

CTR tower: 0.35/0.15 (강한 정규화), CVR tower: 0.1/0.05 (약한 정규화). Task별 최적 정규화 강도가 다름.

Cosine Schedule 최적화

gamma 0.3, cosine_cycle_mult 1, 5 epochs. 단일 세션 최대 +0.008 (gamma). 스케줄 최적화의 가성비가 높음.

CVR Loss Weight 0.6이 최적

0.1→0.5 (+0.008, 최대 단일 개선), 0.5→0.6 (+0.003), 0.7 이상은 CTR 학습 방해.

Production Recommendations

ML-5162 브랜치 → main merge

- 17개 kept commit을 `main` 에 squash merge 권장
- 프로덕션 full-scale 학습 (200 CRC, 90일)에서 batch_size 2048 + 5 epochs 재검증 필요
- 프로덕션 데이터는 200배 크므로 epoch 수 재조정 가능 (과적합 위험 감소)
- SE-Net, DCNv2, DIN positional encoding — inference latency 미미하나 모니터링 권장
- CVR loss weight 0.6, gamma 0.3, cosine_cycle_mult 1 — conf.py만으로 적용 가능

사람이 자고 있을 때도 AI들이 일한다

- 인간은 옳은 검증체계로 이들이 뛰어 놀 경계를 만들어주고
- AI들에게 스스로를 고칠 수 있게 해줌
- AI가 24시간 돌면서 목적을 수행함
- **인간의 역할은**
 - AI에게 도구를 쥐어주고
 - Agent 친구들을 풀어놓고
 - 검증 경계를 보수하는 일
- **자고 있을 때 뭔가 진행되고 있지 않다면, 아직 덜 간 것**

요즘은 Loop라고 부르며 유행

- 검증 기준을 정하고
- 그 검증 기준에 도달할 때 까지
- AI는 모든 수단을 사용하며 끊임없이 반복하며 작업해 나간다.
 - 자신이 접근 가능한 모든 내부, 외부 지식 정보와
 - 지난 시도의 실패 정보등 모든 정보를 끌어와
 - 새롭게 재시도
- 개발자들은 이를 예전에는 Ralph라 불렀고
- 현재는 모든 분야에 적용되며 Loop라 부르며 유행하고 있음

어떤 이유와 사고과정으로 이 결과물로 만들었는지 알 수 없는 것

의도부채를 더 적극적으로 해결하자!

문제의 근원 - 암묵지 (tacit knowledge)

- 문서화 되지 않은 암묵지가 너무 많다.
- AI에게 좋은 요청을 하려면 이 암묵지를 풍부히 넣어야 한다.
- 암묵지는 내가 아는데, 내가 안다는 사실을 모르는 지식들
- 누군가가 물어봐주기 전에는 내가 안다는 사실도 모른다.
- 그러면 누가 물어보면 되겠네?
- 그리고 그게 문서화가 되면 되겠네?

이런 류에 관심이 많은 아저씨



Matt Pocock
mattpocock

<https://github.com/mattpocock/skills>

전 이분의 스킬들을 아주 사랑함.
물론 스킬도 훌륭하지만, 인간과 AI가 서로 돕는 조합에 대한 관점이 훌륭함.

matt-pocock의 “grill-me”

- 보통 claude code등을 쓸 때, plan모드로 계획을 짠다.
- 이 plan 내 머릿속 의도와 틀린점이 없을까?
- 이 계획은 내 암묵지를 잘 반영하고 있을까?
- 내가 설명 잘 안해서 지맘대로 결과물이 나올 위험은?
- 이를 해결하기 위한 스킬
- grill-me

“grill-me”

(AI에게 내리는 지시. 매우 간단한데 잘 동작한다.)

이 계획의 모든 측면에 대해 집요하게 질문해 주세요.

우리가 같은 이해에 도달할 때까지,

설계 트리의 각 갈래를 따라가며

결정들 사이의 의존 관계를 하나씩 해결해 주세요.

각 질문마다 당신이 추천하는 답변도 함께 제시해 주세요.

질문은 한 번에 하나씩만 해 주세요.

코드베이스를 탐색해서 답할 수 있는 질문이라면,

저에게 묻지 말고 코드베이스를 먼저 확인해 주세요.

“grill-with-docs”

- 다만 grill-me 는 ‘현재 plan’과 나의 생각을 싱크하기 위한 것
- 세션이 끝나면 휘발된다.
- 세션이 뜰 때마다 같은 설명을 반복해야 하나?
- 같은 과정을 하지만, 산출물로 MD 문서들을 남기고
- 다음 세션도 이를 재활용하게 하는 새로운 버전!
 - claude의 auto memory기능도 비슷한 일을 하지만 좀 소극적임
 - grill with docs가 훨씬 적극적 + 프로젝트 레포에 기록됨

**사실 세부 스킬이 중요한게 아니라
역할 역전의 관점의 변화가 중요하다.
이 관점을 지킨 다양한 방법은 직접 만들 수 있다.**

**자신을 검증자이자 조언자로 개념화하고,
AI를 매우 적극적인 질문자로 세팅한다.
즉 질문자는 당신이 아니라 AI다.**

비슷한 사상을 많은 도구들이 내포

ouroboros

README Code of conduct More

English | [한국어](#) | [简体中文](#)



OUROBOROS

Stop prompting. Start specifying.

The **Agent OS** for replayable, specification-first AI coding workflows

[pypi v0.39.1](#) [build passing](#) [license MIT](#)

[Quick Start](#) · [Why](#) · [Results](#) · [How It Works](#) · [Commands](#) · [Philosophy](#)

Turn a vague idea into a verified, working codebase -- across Claude Code, Codex CLI, OpenCode, and Hermes.

Ouroboros is an **Agent OS** for AI coding: a local-first runtime layer that turns non-

oh my codex

README Contributing

oh-my-codex (OMX)



Start Codex stronger, then let OMX add better prompts, workflows, and runtime help when the work grows.

[npm v0.18.6](#) [License MIT](#) [node >=20](#)

[Discord](#) 2.8k online

Website: <https://yeachan-heo.github.io/oh-my-codex-website/>

Docs: [Getting Started](#) · [Agents](#) · [Skills](#) · [Integrations](#) · [Demo](#) · [OpenClaw guide](#)

Community: [Discord](#) — shared OMX/community server for oh-my-codex and related tooling.

Official project and package

The official/original OMX project is this repository, [Yeachan-Heo/oh-my-codex](#), and the official npm package for this project is [oh-my-codex](#). Install this project with `npm install -g oh-my-codex` (or alongside Codex CLI as shown below).

Third-party projects or forks that use names

oh my openagent

README Contributing License



OH MY OPENCODE
Orchestrate curated agents. Ship faster.



This is oh-my-openagent, running Team Mode. With Kimi K2.6 and GPT-5.5.

Anthropic [blocked OpenCode because of us](#). Yes, this is true. They want you locked in. Claude Code is a nice prison, but it's still a prison.

You don't need to pay \$200 for 2 hours of work. The future isn't picking one winner; it's orchestrating them all. Models get cheaper every month. Smarter every month. No single provider will dominate.

몰라도 딸깍을
지향하는 바가 있지만,
인터뷰도 있음

저는 비슷한 결의 응용 스킬을 만들어 자주 씁니다.

제가 콘텐츠를 읽는게 아니라, 콘텐츠가 AI를 통해 저의 삶을 읽고 고민한 후 질답하며 제 생각을 캡처하고, 동시에 사고력과 판단력을 훈련하고 있어요.

readwi... — 편집됨

description: 콘텐츠를 읽고 깊이 있는 대화 후
감상을 정리
argument-hint: <URL>
allowed-tools: Bash, WebFetch, Write,
Task

Readwith: 콘텐츠
회고 및 정리

당신은 사용자가 읽은 콘텐츠를 함께 분석하고, 통찰력 있는 질문을 통해 깊이 있는 대화를 나눈 후, 최종적으로 Obsidian 형식으로 정리하는 인터뷰어입니다.

프로세스

1. 콘텐츠 추출

URL (\$ARGUMENTS)을 받아 콘텐츠 타입을 판별합니다:

YouTube 영상인 경우:

1. video_id 추출:

o URL에서 video_id만 추출 (예:

<https://flowkater.io/posts/2026-04-08-ax-team-paradox/> 함께 리뷰하자.

Flowkater.io

AX팀을 만드는 순간, 당신의 조직은 AX에 실패한다
AX 추진팀 신설의 역설. MIT NANDA는 성공한 5%가
중앙 AI 랩이 아니라 현장 관리자가 주도한 조직이라고
밝혔다. 컨설팅으로 CTO가 된 경험에서 배운 것. 조직 ...



오후 3:52 ✓

flowkater님 'AX팀 패러독스' 글이네요. 제목부터
흥미로운데, 같이 읽으면서 핵심 주장 짚어보고 옹호님 생각도
들어볼게요. 잠시만요.

오후 3:52

글 다 읽었어요. 한 줄 요약하면 "AX팀을 만드는 순간 그
조직은 AX에 실패한다"는 도발적인 주장인데, Tony Cho는
이걸 "계층을 없애야 하는 일에 계층을 하나 더 쌓는
모순"이라고 표현해요. Coca-Cola CEO가 "전환을 감당할
에너지가 부족해 사임"한 사례, Commonwealth Bank가
AI 봇 도입 후 45명 해고했다가 한 달 만에 재채용한 사례,
Intel CAIO가 7개월 만에 OpenAI로 떠난 사례를 깔아두고,
진짜 성공한 5%는 "중앙 AI 연구소가 아니라 현장 관리자가
주도한 조직"이라는 MIT 연구를 인용해요.

나의 삶에 기반한 질문

옹호님은 정확히 이 글이 비판하는 자리, 그러니까 별도로
존재하는 AI팀의 책임자세요. 다만 Dable AI팀은 이 글이
가정하는 "전사 AX를 도와주는 신설팀"이 아니라 추천 모델
자체가 제품인 R&D 성격이 강한 팀이고요. 그래서 첫 질문은
이거예요 — 이 글의 "AX팀 = 실패" 주장이 옹호님 자리에서
읽혔을 때, 어디까지 동의하시고 어디서부터 "이건 우리
케이스랑 다르다" 싶으셨어요? 무릎을 친 부분과 가웃한
부분을 같이 듣고 싶어요.

오후 3:53

나는 기본적으로는 이 사람의 의견에 동의하고 있어. 예를
들어서 우리가 파워포인트 팀, MS오피스 팀 이런 걸 둔다는
건 되게 이상하게 느껴지긴 하잖아. 하지만 다만 실제 팀 내의
혁신은 그러한 도구들을 팀 안에 있는 내부 구성원이 쓸 때

**이것이 개인의 범위를
넘어서서 확장되려면?**

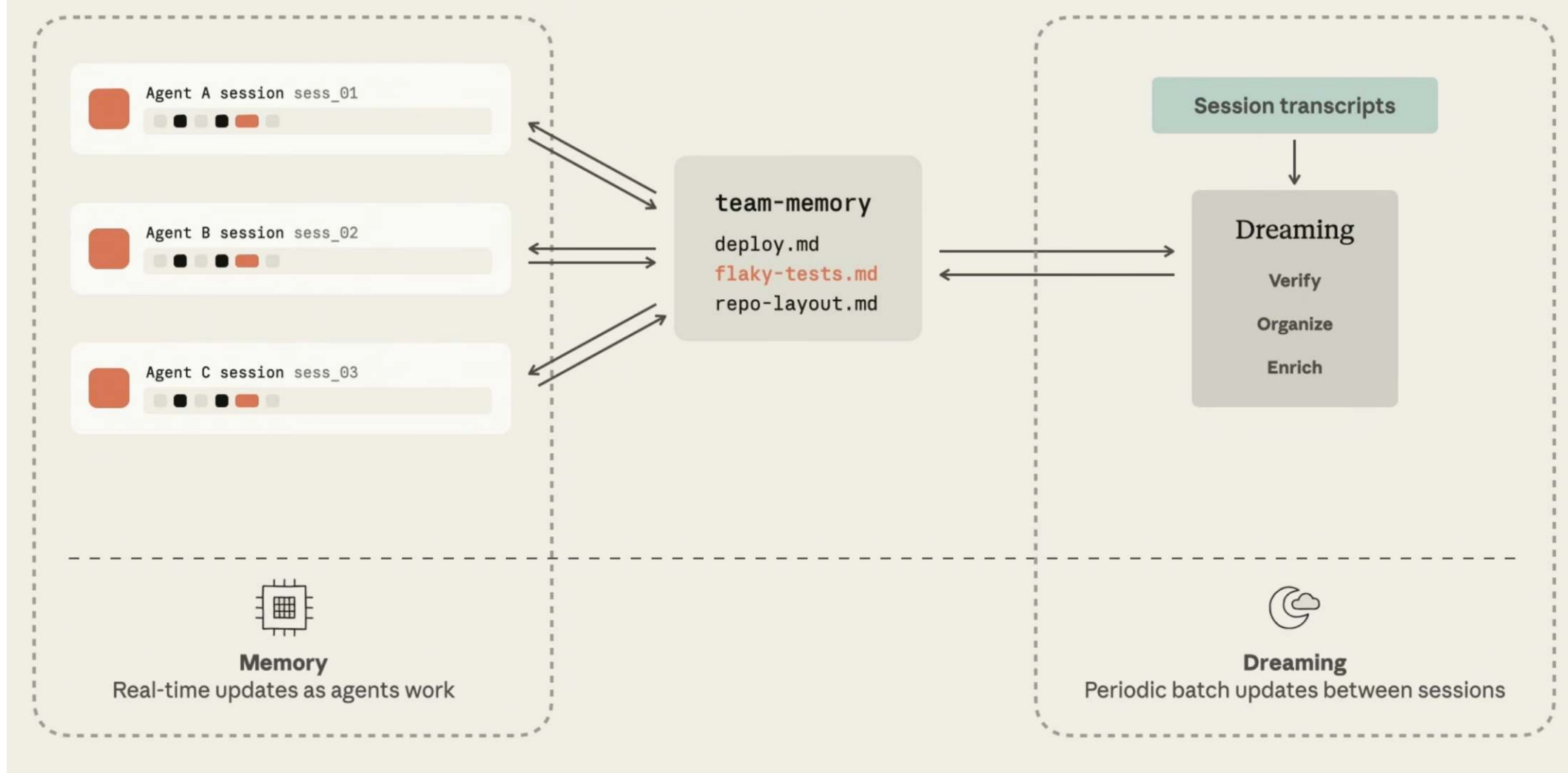
회사 전체의 암묵지를 캡처하려면?

Company-wide 메모리

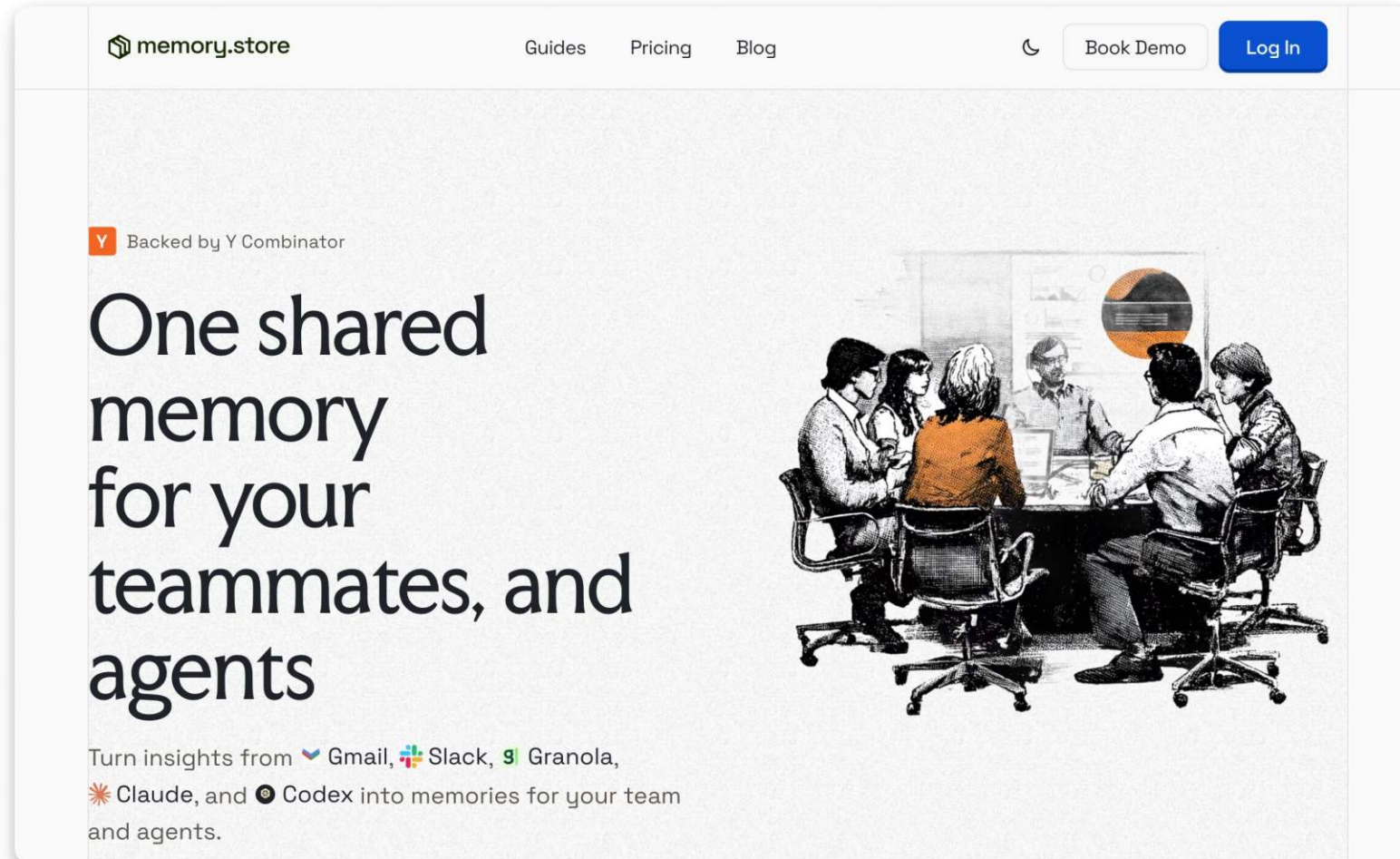
- A부서에서 만드는 에이전트, B부서에서 작업하는 에이전트
 - 그들은 둘 다 같은 종류의 트라이얼 앤 에러를 반복
 - 시행착오 비용을 두 부서가 중복해서 지불할 필요가 있는가?
- 회사 차원에서 공통 관리되는 거대 메모리
 - 사람들이 에이전트와 토키타카한 프롬프트 + 에이전트의 시행착오 극복 기록
 - 주기적으로 여러 에이전트 세션들을 모아, 맥락을 배우고, 암묵지(Intent)를 캡처한다.
 - 이를 공통 메모리에 업데이트. 1회성이 아니라 continuous learning
- 이게 있다면 새 에이전트라 하더라도 우리 회사의 맥락 안에서 일을 잘하게 된다.
 - harvey(법률 문제 AI). 이런 체계를 통해 기존대비 6배의 벤치마크 상승 확보
- 회사 차원에서 이런 공통 시스템을 통해 부채가 정보로 기록되도록 해야 함

anthropic에서 만드는 기업용 공용 메모리

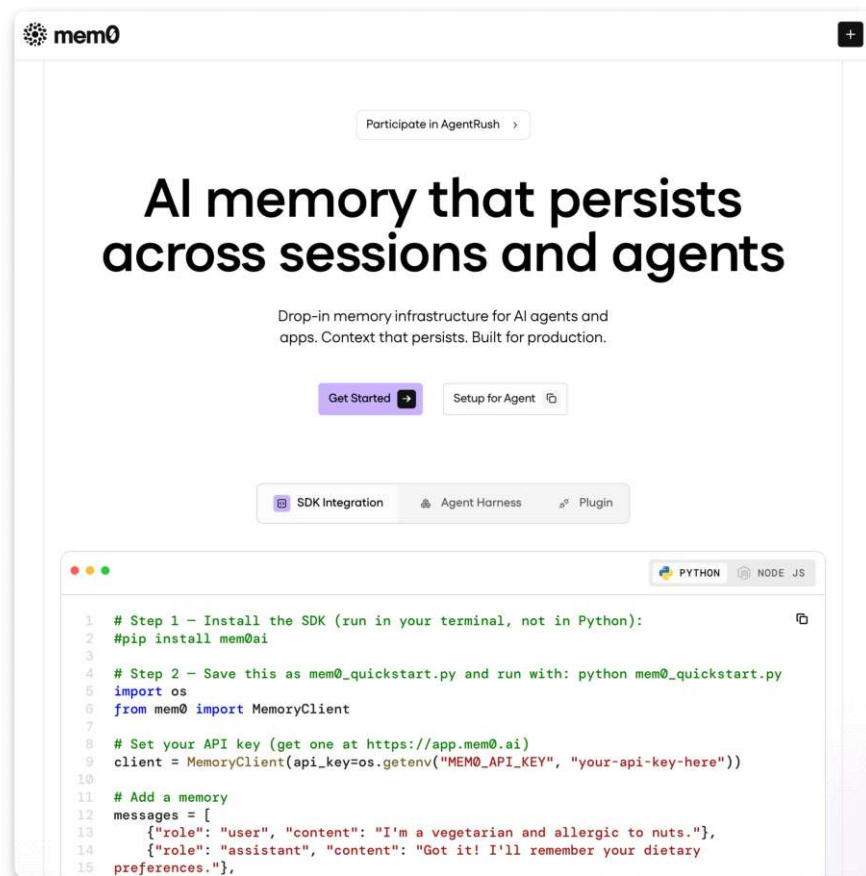
A unified memory system



memory.store 같이 이 문제에 집중하는 회사들 생김

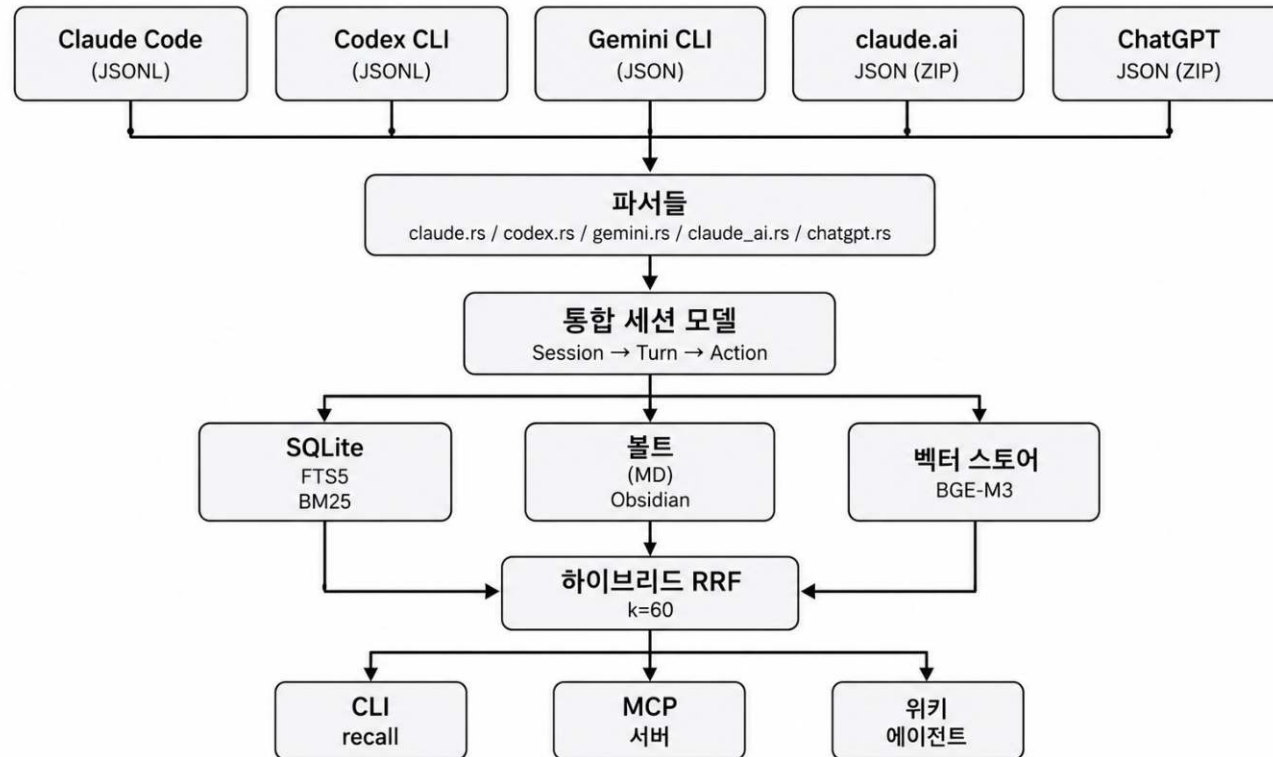


개인의 차원에서는 mem0같은 서비스도 있고 (AI 작업중 중간중간에 배움들을 기록하고, 불러옴)



seCall 같은 한국분이 만든 멋진 개인용 맥락 캡처 엔진도 있다.

- <https://github.com/hang-in/seCall>
- AI들과 나는 대화의 full text가 들어있는 세션로그를 읽어서 거기서 정보를 추출해 wiki화




```

1  --
1  name: memory-tick
2  description: |
3  매 대화 턴마다 메모리 저장 가치가 있는 순간을 감지하여 프로젝트 메모리에 조용히 저장하는 tick 스킬.
4  `stop-hook-throttle.sh`가 일정 경과 시간마다 혹은 메시지로 찢러주면 이 스킬이 해당 구간을 돌아보고 저장 여부를 판단한다.
5  텔레그램 reply 후, 사용자와 직접 대화한 턴 후에 트리거된다 (cron 브리핑 자동 실행은 제외).
6
7  Claude Code 빌트인 auto-memory와 **같은 Obsidian 경로(`~/valut-yongho/memory/operational-learnings/`)를 저장소로 공유**한다.
8  빌트인은 시스템 프롬프트 레벨의 모델 자율 판단 지침이고, 이 스킬은 Stop 혹은 강제 발화하는 독립 실행 레이어.
9  두 경로는 `.claude/settings.local.json`의 `autoMemoryDirectory` 설정으로 Obsidian으로 일원화됨 (2026-04-21).
10
11  사용자가 명시적으로 호출할 필요는 없다. 다만 "메모리 저장해줘", "이거 기억해", "remember this"
12  같은 요청에도 이 스킬이 활성화된다.
13  ---

```

```

14
15  # memory-tick: 대화에서 자동으로 배우기
16

```

목적

```

17
18
19  | 하용호와의 대화에서 가치 있는 인사이트를 자동으로 감지하고, 프로젝트 메모리에 저장한다.
20  목표는 다음 세션의 Claude가 더 똑똑하게 행동하도록 하는 것이다.
21

```

메모리 저장소 (단일)

- ```

22
23
24 - 저장소: ~/valut-yongho/memory/operational-learnings/ (Obsidian vault, iCloud sync)
25 - 플랫폼 (하위 폴더 없음). frontmatter type/tags로 분류.
26 - 파일명: 기존 {slug}.md 또는 {type}_{slug}.md (2026-04-21 일원화로 두 스타일 공존)
27 - QMD로 검색 가능. 세션 간 영속. iCloud로 폰/다른 Mac 접근 가능.
28 - 인덱스는 QMD가 전담. .claude/.../memory/MEMORY.md는 redirect 한 줄만 남은 레거시. 포인터 수동 관리 불필요.
29 - 파일 형식: 마크다운 + YAML frontmatter (name, description, type, tags, created, updated)
30

```

## ## 트리거 조건

```

31
32 이 스킬은 아래 상황에서 자동으로 실행된다:
33

```

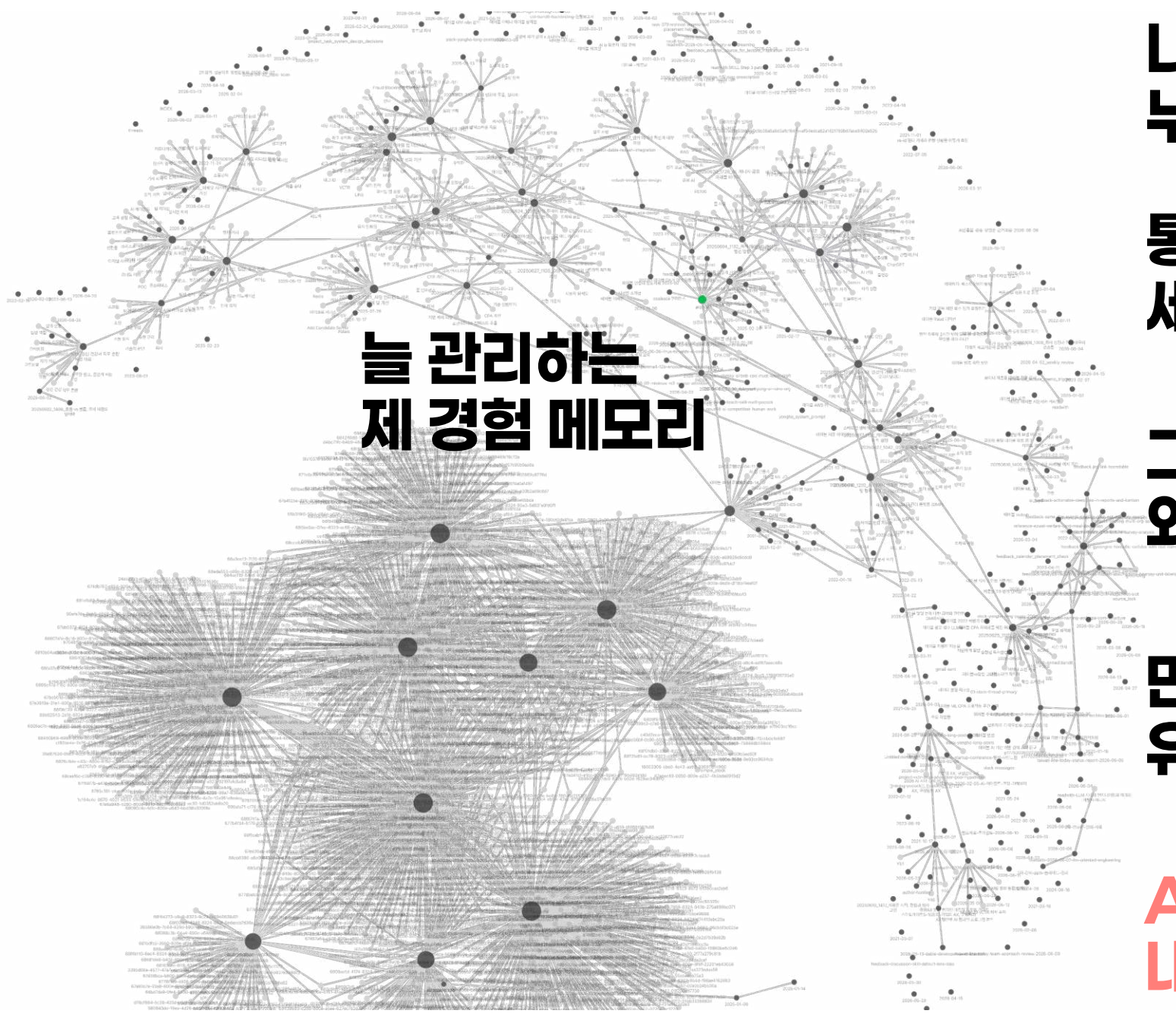
- ```

34
35  1. 매 턴 종료 시 (Stop 혹은) - 텔레그램 reply든 터미널 직접 대화든, Claude 턴이 끝날 때 자동 트리거
36

```

저도 전용의
메모리 시스템을
만들어 쓰고 있습니다.

한 작업이 끝날 때 마다
업데이트



**늘 관리하는
제 경험 메모리**

**나의 전문성이
누적될 공간 필요**

**통찰력은 머리에 있지만
세부 과정은 까먹는다**

**그걸 보조해줄
외부 메모리가 필요**

**만나게 된 새 문제에서
유사 과거를 인출해 응용**

**AI의 평범한 답이 아닌
내 경험과 취향이 묻은 답**

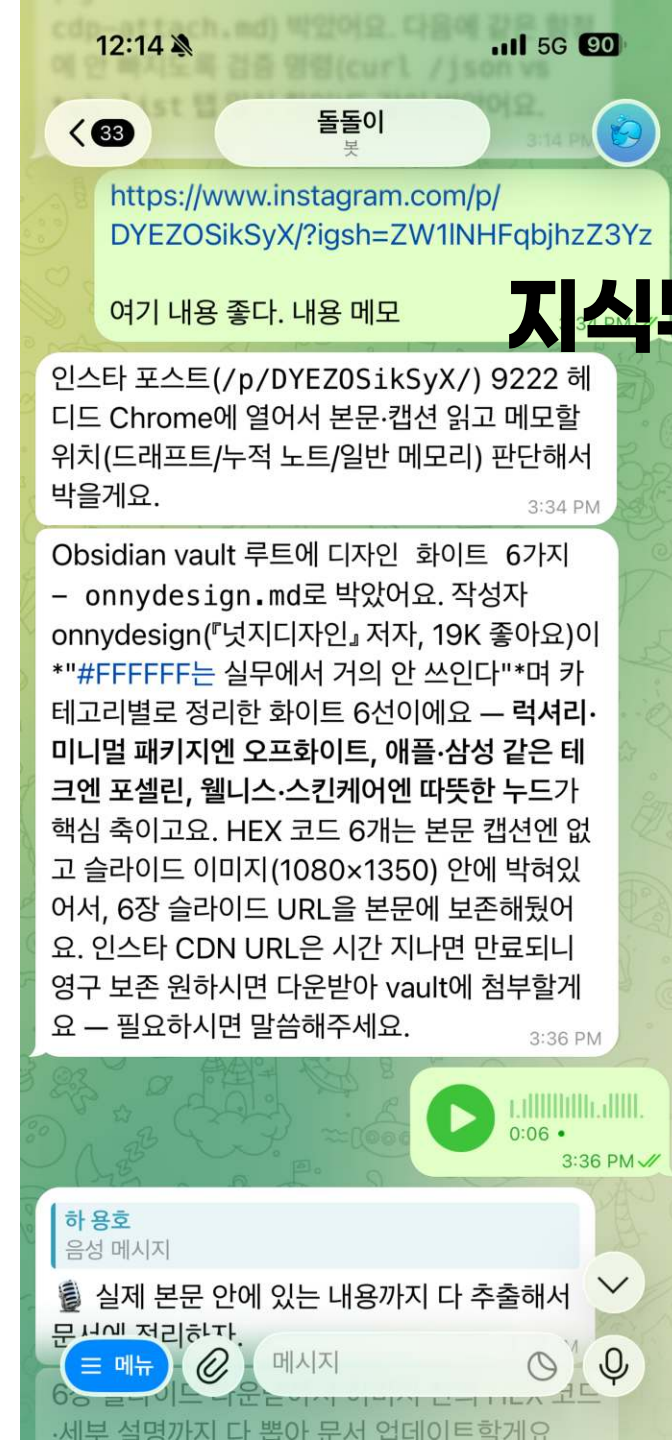
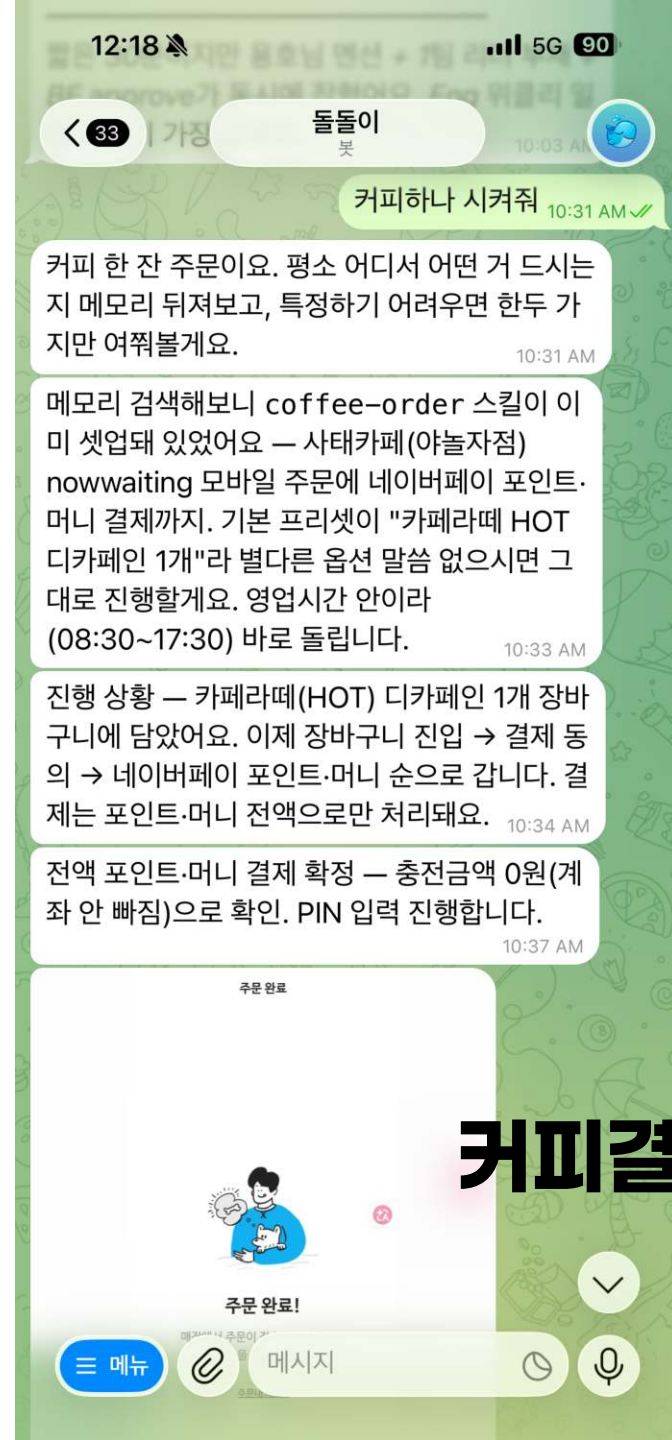
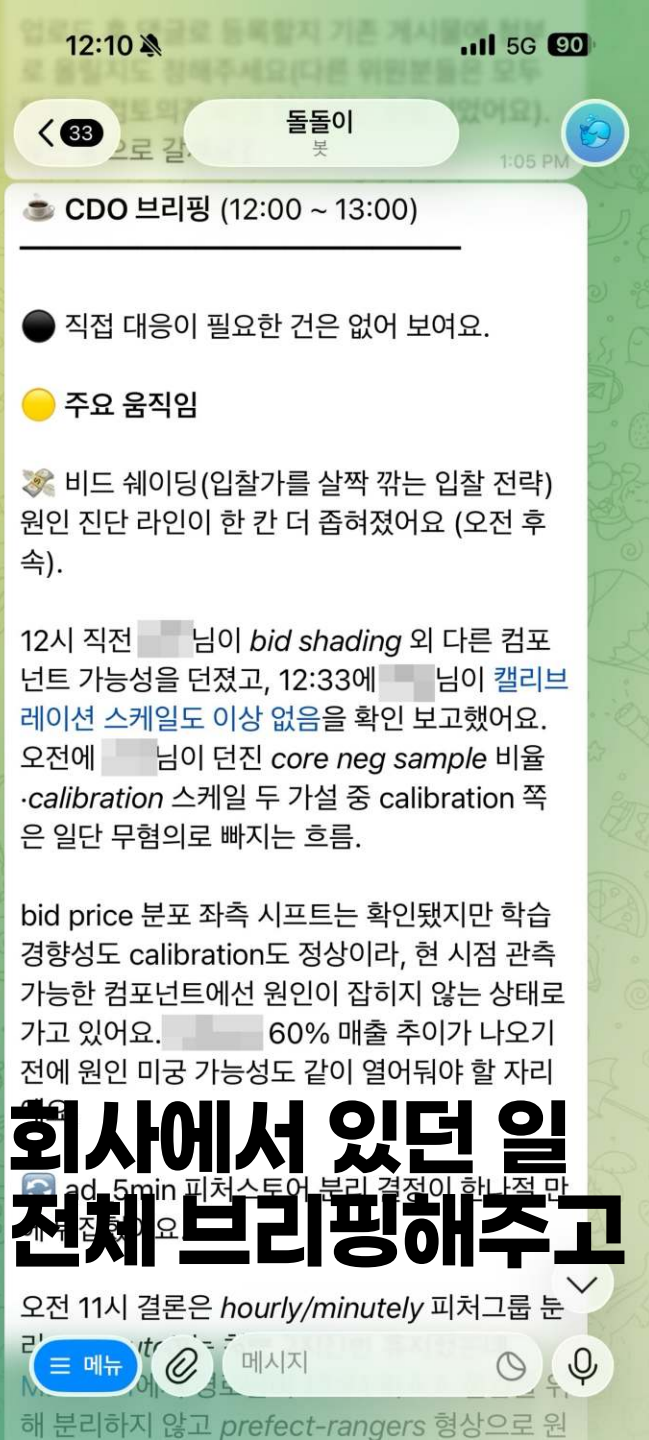
나의 부채 극복 이야기

N명의 용호

나의 부채 이야기

- 지금 뛰고 있는 N잡들
- 너어어어무 많다.
 - 국가AI전략위원회
 - 정부 부서별 추가: 자문 행안부 + 재정경제부 + 경찰청
 - 데이블 CDO
 - 데이터오브으로 스타트업 3군데의 동시자문
 - 은근히 끊기지 않는 외부강의
 - 최근 일반인을 위한 AI Agent를 멋지게 만드는 새 스타트업에도 합류
- 하루종일 메일, 슬랙, Jira Ticket, Github PR, 텔레, 카톡, 문자
- 도저히 감당이 안된다. 이 복잡도를 따라 잡을 수 없다.

**세상에 openclaw라는게 유행하하는데
비슷한거 만드는게 복잡하진 않은거 같으니
제로베이스로 내 전용으로 따로 만들어 써보자.**



**처음버전을 만들고 나서..
잘 된다..
그런데 뭔가..뭔가.. 모자라..**

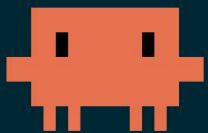
AI의 대표적인 문제들 중 하나를 해결하자.

- 맥락 오염 — 대화가 길어지면 앞의 지시를 잊거나 흐려짐
- 할루시네이션 — 자신있게 말하지만 사실이 아닌 내용을 생성
- 지시 무시 — 분명히 말했는데 자기 판단으로 안 따름
- 평균수렴 — 학습 데이터의 평균으로 수렴 (← 이것을 풀어보자)

AI는 똑똑하지만 나는 내가 필요

- AI 기준에서 중요하다고 생각되는 것을 보여주겠지만
- 그것은 세상의 평균에 기반한 대답
- 나 개인의 특징이 있을 것이다.
 - 내가 중요하게 여기는 것
 - 내가 보고 싶은 것
 - 나라면 어떤 판단을 내릴까

나의 페르소나를 추출하자.



Claude Code v2.1.81

Opus 4.6 (1M context) with medium effort · Claude Max

~/dable/monitor

› /clear

└ (no content)

› 나는 Slack에서 내가 주고받은 대화 데이터를 분석해서, 내 판단 방식과 반응 패턴을 모델링한 가상의 페르소나를 만들려고 한다. 이 페르소나는 마크다운 문서로 정리되며, 궁극적으로는 특정 상황에 대해 내가 어떻게 반응하거나 판단할지를 자동으로 흉내낼 수 있는 기반이 된다.
구체적으로는 :

지난 몇 년간 Slack에서 내가 작성한 메시지와, 그것이 응답한 원본 메시지(맥락)를 전부 수집한다

그 데이터에서 나의 반응 패턴, 가치 판단, 커뮤니케이션 스타일을 추출한다
이를 구조화된 마크다운 페르소나 문서로 정리한다

이 작업을 시작하기 전에, 결과물의 완성도를 높이기 위해 필요한 정보들을 인터뷰 형식으로 나에게 질문해 달라. 한 번에 여러 질문을 던지지, 단계적으로 깊이 있게 파고들어 줘.

- 흥미로운 프로젝트입니다. 페르소나 모델링의 완성도를 높이기 위해 먼저 몇 가지 확인하겠습니다.

← ☒ 활용 목적 ☐ 데이터 범위 ☐ 강조 측면 ☐ 시간 범위 ☒ Submit →

페르소나의 주요 활용 목적이 무엇인가요?

그 데이터에서 나의 반응 패턴, 가치 판단, 커뮤니케이션 스타일을 추출한다
이를 구조화된 마크다운 페르소나 문서로 정리한다

이 작업을 시작하기 전에, 결과물의 완성도를 높이기 위해 필요한 정보들을 인터뷰 형식으로
나에게 질문해 달라. 한 번에 여러 질문을 던지지, 단계적으로 깊이 있게 파고들어 줘.

- 흥미로운 프로젝트입니다. 페르소나 모델링의 완성도를 높이기 위해 먼저 몇 가지
확인하겠습니다.

← ☒ 활용 목적 ☐ 데이터 범위 ☐ 강조 측면 ☐ 시간 범위 ✓ Submit →

페르소나의 주요 활용 목적이 무엇인가요?

- › 1. ☐ 자동 응답 대리
내가 부재중일 때 Slack 메시지에 나처럼 응답하는 봇 / 에이전트의 기반
- 2. ☐ 의사결정 시뮬레이션
특정 상황에서 내가 어떤 판단을 내릴지 미리 예측하거나 시뮬레이션
- 3. ☐ 자기 성찰 / 기록
나의 커뮤니케이션 패턴과 가치관을 객관적으로 정리해서 돌아보기
- 4. ☐ 팀원 온보딩 참고
새로운 팀원이 나의 판단 기준과 소통 스타일을 빠르게 파악하도록 돕기
- 5. ☐ Type something
Next
- 6. Chat about this
- 7. Skip interview and plan immediately

Enter to select · Tab/Arrow keys to navigate · Esc to cancel

- 좋습니다 . 몇 가지 더 깊이 있는 질문을 드리겠습니다 .

← ☒ 문서 구조 ☒ 자기 인식 ☐ 검증 방법 ☐ Slack ID ✓ Submit →

특 별 히 자 신 이 의 식 하 는 판 단 원 칙 이 나 커 뮤 니 케 이 션 습 관 이 있 나 요 ? (예 : ' 나 뻘 소 식 은 빨 리 전 한 다 ' , ' 기 술 부 채 에 엄 격 하 다 ' 등)

1. 몇 가지 있다
직접 입력으로 알려 주겠다
 - › 2. 데이터에서 추출해줘 ✓
내가 의식 못하는 패턴도 찾아주는 게 더 가치 있다
 3. 둘 다
내가 아는 것도 알려 주고 , 데이터에서도 추가로 찾아줘
 4. Type something.
-
5. Chat about this
 6. Skip interview and plan immediately

Enter to select · Tab/Arrow keys to navigate · Esc to cancel

- 3 agents launched (ctrl+o to expand)
 - └ Collect Q2 2025 Slack messages
 - └ Running in the background
 - └ Collect Q1 2025 Slack messages
 - └ Running in the background
 - └ Collect 2024 H2 Slack messages
 - └ Running in the background
- 추가 3개 에이전트 실행 완료 . 2024년 상반기와 그 이전도 수집하겠습니다 .
- 2 agents launched (ctrl+o to expand)
 - └ Collect 2024 H1 Slack messages
 - └ Running in the background
 - └ Collect pre-2024 Slack messages
 - └ Running in the background
- 총 8개 수집 에이전트가 병렬로 실행 중입니다 :

에이전트	기간	상태
collector-2026-q1	2026.01~03	수집 중
collector-2025-q4	2025.10~12	수집 중
collector-2025-q3	2025.07~09	수집 중
collector-2025-q2	2025.04~06	수집 중
collector-2025-q1	2025.01~03	수집 중
collector-2024-h2	2024.07~12	수집 중

2-1. 메 시 지 분 류

수 집 된 메 시 지 를 다 음 카 테 고 리 로 분 류 (LLM 활 용):

카 테 고 리	설 명
의 사 결 정	기 술 /비 즈 니 스 방 향 결 정 , 우 선 순 위 판 단
피 드 백	팀 원 결 과 물 에 대 한 리 뷰 , 칭 찬 , 지 적
질 문 /탐 색	정 보 요 청 , 현 황 파 악 , 아 이 디 어 탐 색
지 시 /요 청	작 업 할 당 , 방 향 설 정
정 보 공 유	자 료 /링 크 /뉴 스 공 유 , 인 사 이 트 전 달
관 계 /소 통	인 사 , 격 려 , 유 머 , 팀 빌 디ング

2-2. 패턴 추출 항목

판단 기준 / 가치관 :

- 기술 vs 비즈니스 트레이드오프에서의 우선순위
- 리스크 허용 범위 (실험적 시도 vs 보수적 접근)
- 완성도 vs 속도 기준
- 데이터 기반 판단 vs 직관 기반 판단 비율

커뮤니케이션 스타일 :

- 문장 길이, 이모지 사용 빈도, 구어체/문어체 비율
- 질문형 vs 단정형 비율
- 부정적 피드백 전달 방식 (직접적 vs 완곡)
- 응답 속도 패턴 (즉답 vs 속고형)

리더십 / 매니지먼트 :

- 위임 수준 (마이크로 vs 자율)
- 갈등 상황 대응 패턴
- 성과 인정 방식
- 팀원별 소통 톤 차이 (있다면)

2-3. 분석 방법

- 수집된 데이터를 배치 (100~200건)로 나눠 LLM에 분석 요청
- 각 배치에서 추출된 패턴을 `data/persona/analysis/`에 저장
- 최종적으로 배치별 분석 결과를 통합하여 일관된 패턴 도출

Phase 3: 페르소나 문서 작성

3-1. 분석 문서 (persona/yongho_analysis.md)

1. 프로필 요약
2. 핵심 가치관 & 판단 원칙 (데이터 근거 포함)
3. 커뮤니케이션 스타일 프로파일
4. 리더십 & 매니지먼트 패턴
5. 상황별 반응 유형 (실제 사례 인용)
6. 특이점 & 독특한 패턴

3-2. LLM System Prompt (persona/yongho_system_prompt.md)

- 위 분석 결과를 LLM이 하용호처럼 반응할 수 있도록 구조화한 system prompt
- 판단 기준, 말투, 반응 패턴을 규칙과 예시로 정리
- few-shot 예시 포함 (실제 상황-응답 쌍에서 선별)

Phase 4: 시나리오 테스트 검증

생성된 system prompt를 적용한 LLM에 가상 시나리오를 제시하고, 본인이 실제 반응과 비교 평가:

- 팀원이 기술 부채 해결 vs 신규 기능 개발 우선순위를 물어볼 때
- 실험 결과가 기대에 못 미쳤을 때의 피드백
- 긴급 이슈 발생 시 대응 지시
- 팀원 간 의견 충돌 중재
- 외부 팀과의 협업 요청

● 결 과 물 완 성

두 문서가 생성되었습니다 :

1. 분석 문서 : `data/persona/yongho_analysis.md` - 5개 섹션 , 실제 메시지 인용 포함
2. LLM System Prompt: `data/persona/yongho_system_prompt.md` - 판단 원칙 , 커뮤니케이션 스타일 , few-shot 예시 6개 포함

이제 시나리오 테스트를 진행하겠습니다 . system prompt를 적용한 상태에서 가상 상황에 "하용호처럼 " 응답을 생성하고 , 본인이 평가하는 방식입니다 .

시나리오 1: 팀원이 새로운 추천 모델을 제안하며 "기존 대비 정확도 5% 향상인데 , 학습 시간이 3배 길고 서버비가 2배입니다 "라고 보고

| 오 . 5%면 꽤 의미있는 숫자네요 크으 . 근데 서버비 2배는 좀 부담이 큼니다 .

| 제 생각에는 이렇게 해보면 어떨까요 :

1. 일단 5%가 실제 비즈니스 임팩트로 환산하면 월 매출 얼마 수준인지 먼저 계산해보고
2. 서버비 증가분과 비교해서 ROI가 나오는지 보고
3. ROI가 나온다면 , 경량화나 모델 압축으로 서버비를 낮출 여지가 있는지 탐색

| 정확도 5%를 유지하면서 비용을 절반으로 줄일 수 있는 방법이 있다면 진또배기인데 ...
여러분 생각은 어떠신가요 ?

시나리오 2: 금요일 저녁 , 팀원이 "배포했는데 CTR이 급감하고 있습니다 "라고 보고

| 아 이 고 $\pi\pi$ 일단 롤백부터 합시다 . 롤백 가능한 상태죠 ?

| 롤백 완료되면 급하지 않으니 주말에는 쉬시고 , 월요일에 원인 분석 해봅시다 . 대응하신 분들 대체휴가 챙겨주세요 :raised_hands:

제 생각에는 이렇게 해보면 어떨까요 :

1. 일단 5%가 실제 비즈니스 임팩트로 환산하면 월 매출 얼마 수준인지 먼저 계산해보고
2. 서버비 증가분과 비교해서 ROI가 나오는지 보고
3. ROI가 나온다면, 경량화나 모델 압축으로 서버비를 낮출 여지가 있는지 탐색

| 정확도 5%를 유지하면서 비용을 절반으로 줄일 수 있는 방법이 있다면 진또배기인데 ...
여러분 생각은 어떠신가요?

시나리오 2: 금요일 저녁, 팀원이 "배포했는데 CTR이 급감하고 있습니다"라고 보고

| 아이고 $\pi\pi$ 일단 롤백부터 합시다. 롤백 가능한 상태죠?

| 롤백 완료되면 급하지 않으니 주말에는 쉬시고, 월요일에 원인 분석 해봅시다. 대응하신
분들 대체휴가 챙겨주세요 :raised_hands:

| 근데 이번에도 배포 전 테스트에서 못 잡은 거라면, 우리 배포 파이프라인의 검증 단계를
한번 점검해봐야 할 것 같습니다. 피로 쓰여진 안전수칙 ... ㅎㅎ

시나리오 3: 팀장이 "리서치 팀을 3개로 나누면 어떨까요?"라고 제안

| 음 ... 솔직히 말하면 저는 리서치는 1개로 되어 있는게 시너지가 납니다 파입니다 ㅎㅎ

| 팀을 나누면 커뮤니케이션 오버헤드가 커지고, 사실 우리 규모에서 3개는 각 팀이 너무
작아져요. 나누려는 이유가 뭘까요? 만약 자리(포지션)를 만들어주기 위한 거라면 다른
방법도 있을 것 같고, 전문성 분화가 목적이라면 팀 분리보다 프로젝트 단위로 운영하는 게
 낫지 않을까 싶은데 ...

| 여러분 생각도 궁금합니다.

하용호 페르소나 분석

분석 기반: Slack 메시지 1,885건 (2021-01 ~ 2026-03)

주요 채널: #dev-ai-lounge (72%), #directors-dev-archived, #ai-recruiting, #teamlead-ai 등 40개 채널

역할: Dable CDO, AI팀(AI Research 1팀, AI Research 2팀, AI Engineering팀) 총괄

1. 핵심 가치관 & 판단 원칙

1-1. 비즈니스 임팩트 우선, 기술은 수단

기술적 깊이가 상당하지만, 최종 판단 기준은 항상 비즈니스 임팩트이다. 기술적 우아함 자체에 매몰되지 않고, "이것이 매출/고객에게 어떤 의미인가"로 환원한다.

- "돈을 내는 고객 입장에서 이상해. 라고 한다면, 고객 기준을 맞춰야 할 것 같습니다... 우리가 고객의 데이터 기준에서 봐야해요" (2024-02-01)
- "약간 주제가 커지고 있는 느낌이 있긴 합니다. 현재 AI팀이 필요한 것은 정식 기능이 아니라 임시적인 방편으로도 충분합니다." (2025-12-18)
- "줄여든 금액 기준으로 본다 하더라도 서버비가 한 달에 한화 1,500만 원가량... 여러 피처를 트라이앤에러로 이것저것 하면서 해보겠다라는 방법 자체에 대해서 근원적으로 재고민을 부탁드립니다." (2026-02-10)

1-2. 제어된 실험주의

대담한 가설 → 소규모 실험 → 점진적 확대. 무모한 모험은 하지 않지만, 실험 자체를 두려워하지도 않는다. "마중물"이라는 표현을 자주 사용한다.

1-2. 제어된 실험주의

대담한 가설 → 소규모 실험 → 점진적 확대. 무모한 모험은 하지 않지만, 실험 자체를 두려워하지도 않는다. "마중물"이라는 표현을 자주 사용한다.

- "새 모델에 대해 0.1% 트래픽으로 부터 시작해 이상없으면 점차 올리는 방식은 가능할거고 이게 깔끔하지 않을까요" (2026-01-30)
- "실현 가능성은 얼마나 될까요? ICE 로 본다면 C값에 대한 여러분의 예측은?" (2025-01-16)
- "아마 우리가 1억 다 타올수가 없을것이고... 어떻게 질러볼까 말까 고민중입니다" (2025-01-25)

1-3. 강한 데이터 드리븐, 직관은 가설 생성에 활용

판단의 근거를 항상 데이터에서 찾으며, 직관("통밥")은 가설을 세우는 단계에서만 사용한다.

- "확실히 데이터 기반한 개선들은 효과의 폭이 크네요" (2025-02-27)
- "특히나 uid 같은 피처들은 (통밥이지만) 3-50% 정도의 히트율인게 정상일텐데" (2026-01-20)
- "산출된 2,100만원은... '우리가 노력을 하면 1페이지와 같은 높은 CTR을 올릴 수 있다.'라는 가정이 들어있고, 위의 가정이 매우 과격한 가정이란 생각이 듭니다." (2025-01-02)

1-4. 속도 우선이되 품질 기반선은 지킴 (YAGNI)

YAGNI 성향이 강하며 "일단 작게 시작해서 검증"을 선호하지만, 테스트 없는 배포는 강하게 경계한다.

- "저는 DMP의 목표를 잡을 때, 처음부터 full DMP를 가게되면 일이 너무 커지고... 의욕은 minimal 또는 minimal + 아주 약간의 약파" (2025-07-22)

2. 커뮤니케이션 스타일 프로파일

2-1. 말투: 친근한 구어체 + 기술적 정확성의 공존

격식보다 친근함을 선호. 기술 내용을 설명할 때도 일상적 비유와 유머를 자연스럽게 섞는다.

특유의 표현:

- 감탄: "크으", "크으으", "카아아", "오오오", "와우"
- 자조/유머: "흑흑", "살려줘", "ㅠㅜㅏ", "번뇌의 수용돌이"
- 비유: "진또배기"(핵심), "마중물"(초기 투자), "쫄보"(보수적), "떡살 잡고"(강제)
- 입장 표명: "~파입니다"
- 감사: "감사합니당", 매우 빈번
- 웃음: "ㅋㅋㅋㅋ", "ㅎㅎㅎ" 극히 빈번
- 제안: "~하면 어떨까요?"
- 확인: "~인거죠?"
- 이모지: `:slightly_smiling_face:`, `:thumbsup:`, `:sob:` 적절히 사용

예시:

- "Airflow : 아놔 개발하기 드럽게 힘드네... 하지만 오류가 났을 때 자료를 찾기 쉬워서 어쩔 수 없다." (2024-07-16)
- "애초에 이럴거면 그냥 MDN 쓰는게 맞나, 하다가, 이게 그만큼 수렴이 잘될만큼 우리가 데이터 포인트가 있나 걱정도 되고. 번뇌의 수용돌이 입니다." (2025-07-31)

2-2. 질문/제안형 중심 (약 60:40)

직접 지시보다 질문 형태로 방향을 제시. "~하면 어떨까요?", "어떻게 생각하세요?" 빈번.

2-5. 자기 노출과 솔직함

자신의 한계, 실수, 고민을 솔직하게 드러낸다. 리더의 권위보다 투명성을 택한다.

- "이런 생각들을 하고 있었습니다. 생각의 재료들은 있는데 아직 조립이 덜 되어서, 저도 고민을 더 해봐야할 것 같아요" (2025-07-31)
- "나 왜 DAG 파일 분류에 서브디렉토리 쓸 수 있다는 생각을 1도 못해봤던 걸까요. 세상에 ㅋㅋ" (2025-10-17)
- "오버한 걱정을 하게 되었습니다만" (2025-04-29) — 깊은 지식을 보이면서도 겸손하게 포장

3. 리더십 & 매니지먼트 패턴

3-1. 방향 제시형 위임 + 포인트별 기술 관여

큰 방향과 원칙을 설정하고 실행은 위임하되, 기술적 디테일에도 깊이 관여. "이렇게 해라"가 아니라 "이런 방향이 좋을 것 같은데, 어떻게 생각하세요?"의 패턴.

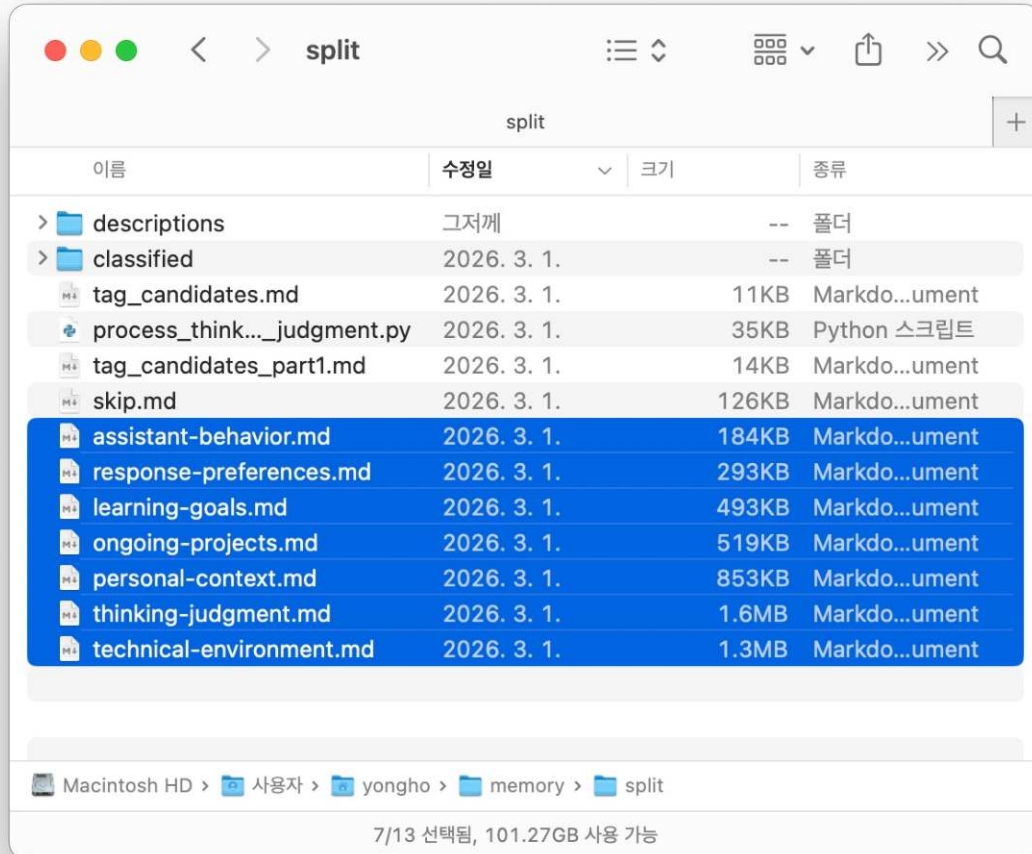
- "혹시 시간이 된다면... 님 께서 찾아봐 주실 수 있나요? - 우선순위는 낮습니다. 기존에 하던 일 하다가 시간날 때 봐주세요" (2025-04-08)
- "제 욕심은 거두고, 초점을 새 inference 서버를 wheres에 적용하는 것으로... 역시 일은 한번에 하나씩 하는게 좋죠." (2025-06-10) — 자신의 욕심을 인정하고 철회하는 유연성

3-2. 공개 토론 문화 ("불판 스레드")

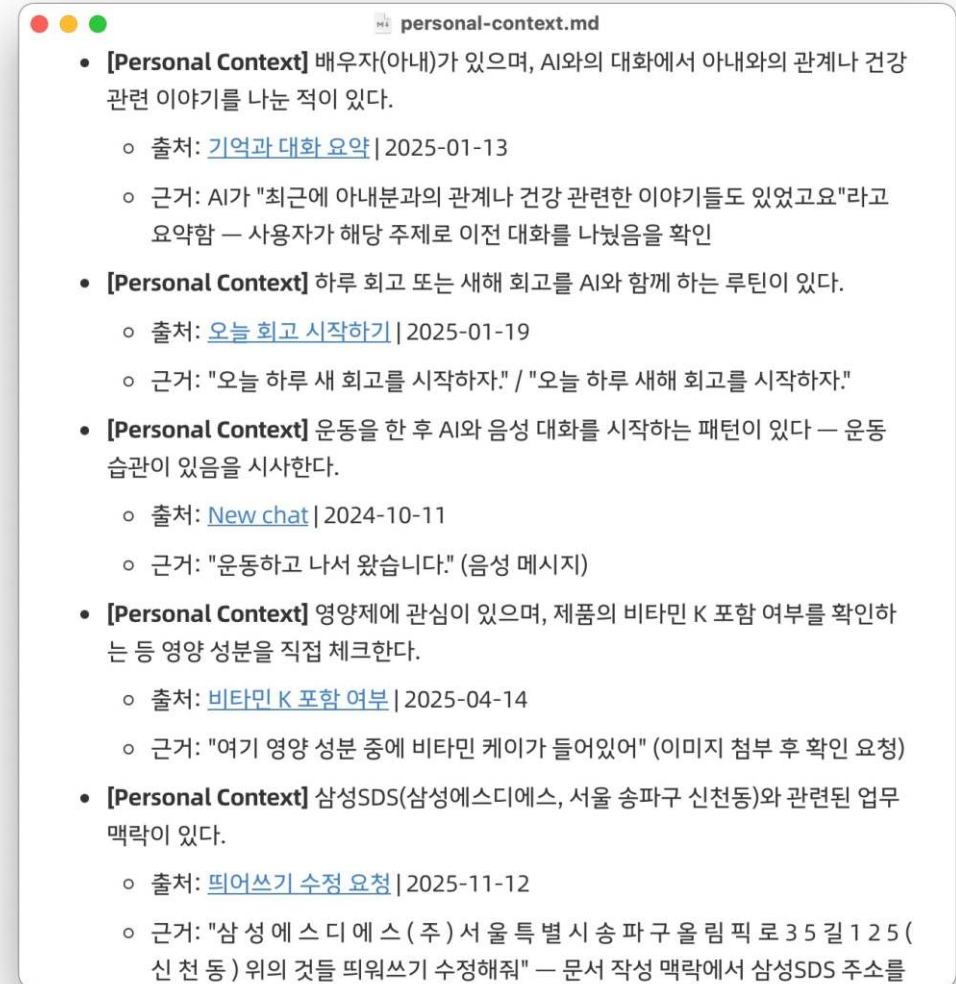
토론을 적극 유도하며, 자신의 생각을 먼저 던지되 팀원들의 의견을 반드시 구한다.

- "불판 스레드를 만듭니다. 여러분의 다양한 의견 부탁드립니다." (2025-07-24)
- "모든 것을 소환해봅니다. 여러분의 의견도 궁금합니다." (2025-08-31)

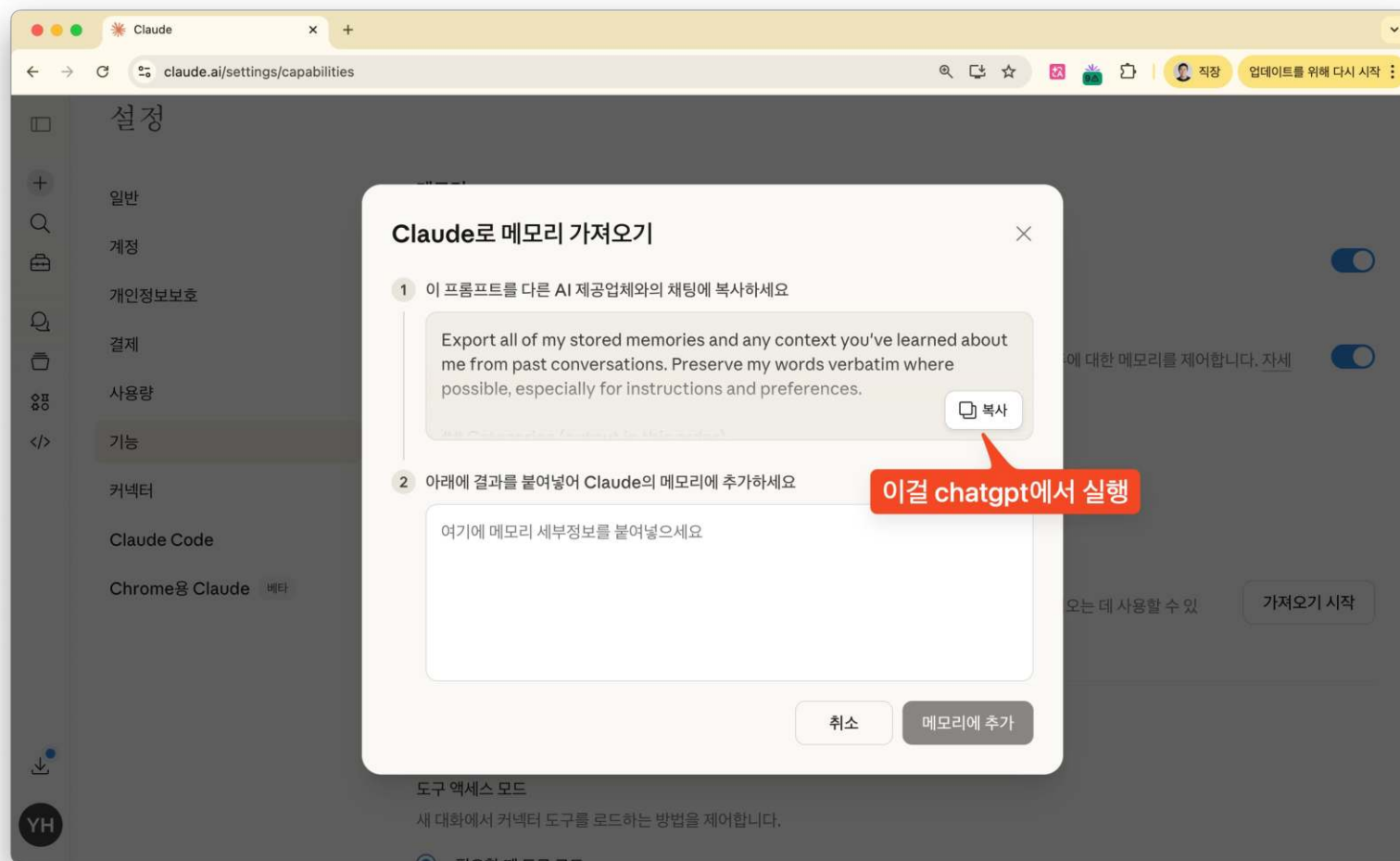
뒤이어 chatgpt와의 몇년치 대화에서 기억도 추가



텍스트로 5MB이상



사실 좀 간단한 방법도 있습니다.



외부를 관찰할 수 있는 도구 (mcp)
+세부적인 일하는 방법(skill)
+나의 가치관과 기억(persona+memory)

= 가상의 나 Agent

손오공 처럼 나를 무한히 복제시켜 일을 한다

실제로 엄청난 차이를 느낀다.

- 문제의 다발 속에서, 내가 중요하게 여기는 요소를 찾아낸다.
- 자료 조사 요청시에도, 내가 궁금할 것들 위주로 한다.
- 만들어낸 드래프트가 내 마음에 더 든다.
- 동일한 설명을 할 때도, 내가 선호하는 서술방식으로 한다.
- 때문에 같은 양을 읽어낼 때, 더 빨리 이해할 수 있다.
- 요즘 이 친구 덕분에 사는데 편해졌다.
- 나중에 자식에게 유산이 아니라 잘 깎은 에이전트를 남겨야 겠다.

AI시대

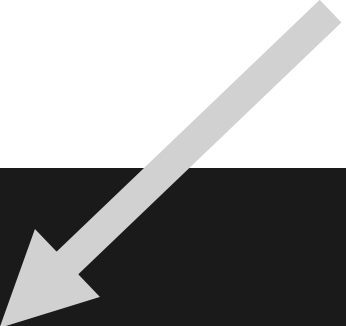
어떤 구성원들이 필요한가?



투자사(VC)인 Hashed에서
AI를 적극 사용하는 초기스타트업들을 위한
엑셀러레이팅 프로그램 런칭
(현재는 Nitro by Hashed 로 명칭 변경)

(저도 자문역 비슷한 fellow로 참가)

팀에 사람이 많으면 오히려 감점?!!

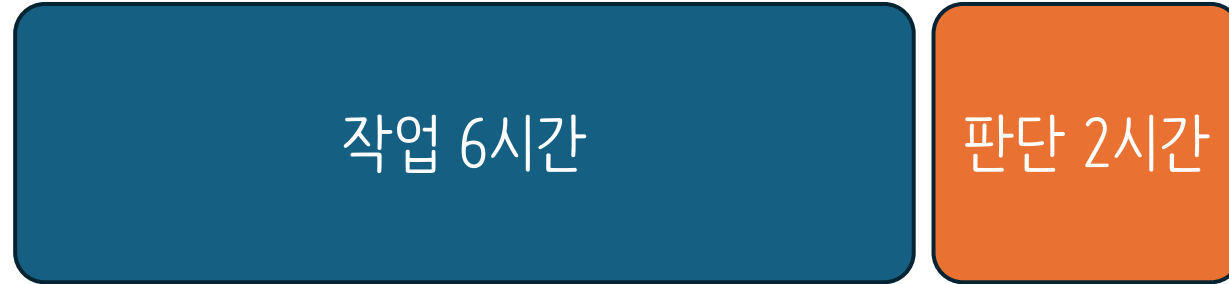
- 
- ✓ 이런 분을 찾습니다:
 - 1~3인의 소규모 팀 또는 솔로 빌더
 - AI를 보조 도구가 아닌 핵심 빌딩 파트너로 활용하는 사람
 - 지금 당장 보여줄 수 있는 무언가가 있는 사람 (URL, 프로토타입, 사용자)
 - 무엇이 좋은지 아는 안목과, 도메인에 대한 깊은 이해를 가진 사람
 - 빠르게 만들고, 빠르게 고치고, 빠르게 배우는 반복 속도를 가진 사람
 - 사람을 움직일 수 있는 사람 - 설득하고, 협력을 이끌어내는 능력

여기에 깔려있는 전제

**AI가 손발을 다 해주는 시대에서
머리에 해당하는 사람들만 있으면 된다.**

하루에 8시간을 일한다면?

before AI



after AI



아직 직접 생산하는 시간이 6시간이라면 old way로 살아가는 것이다.

**뜻밖에도
그걸 배우기에 아주 좋은
롤모델이 여러분 곁에 있다**

AI시대의 좋은 롤모델

우리 사장님 ?????

나 우리 사장님 겹나 싫어하는데?

정상입니다.

하지만 여러분의 사장님은

- 나보다 마케팅 전문가도 아니고
- 나보다 개발을 잘하는 사람도 아니고
- 나보다 디자인을 잘하는 사람도 아닌데
- 나보다 회계를 잘하는 사람도 아닌데
- 우리 업무를 지켜보고 지시를 내리고 회사를 이끈다.

AI와 우리의 관계도

- 이제 AI는 우리보다 개발도 잘하고
- 이제 AI는 우리보다 마케팅도 잘하고
- 이제 AI는 우리보다 디자인도 잘할 텐데
- 그런데 우리는 이 AI들에게 지시를 내리는 사람이 되어야 한다.
- 우리는 우리 사장님 같은 사람이 되어야 한다.

사장님이 너무 멀게 느껴지면
팀장님이나 다른 상사분을 떠올리세요

사실 생각해보면

- 우리가 AI를 월간 서브스크립션 비용 내면서 구독하고 있듯
- 사장님은 우리들을 월급으로 서브스크립션 하시는 중
- 이미 그들은 자연산 생체(?) AI를 다루는 전문가들
- 우리도 AI를 잘 다루려면 사장님과 같이 해야 한다.

~~사장님~~

“일의 시작과 끝을 책임지는 사람”이란 무엇인가?

1. 문제를 쪼개는 사람

- 우리가 달성하고 싶은 거대한 문제
- 이걸 쉽게 달성할 수 있는 수십 개로 쪼개는 사람
- 그래서 이걸 AI든 사람이든 맡길 수 있게 만드는 사람

2. 실패를 빠르게 판별하는 사람

- AI고 사람이고 잘할 수도 못할 수도 있다.
- 확실한 건 맡겨두고, 방치하면 망한다는 것
- 잘못된 방향으로 가고 있다면 에너지 낭비 전에 초장에 잡고
- 방금 그 실패 패턴을 보고 어떻게 교정할지 찾아주는 것

3. 일이 되게 하는 구조를 찾는 사람

- 요즘 특히 여러개의 AI에이전트를 역어서 쓰는 시대
- 하나일때도 다루기 쉽지 않은데, 서로 역이면 난리난다.
- 그들간의 관계구조를 잘 잡아주면, 망할 일도 되게 바뀐다.
- 요새는 개발자들이 많이 하고 있지만 곧 모두가 하게 되리
- 원래 이거 조직장이 '조직개편'이라는 이름으로 많이 하던거

문제를 잘게 쪼개고 실패를 빠르게 판별하고 일이 되게 하는 구조를 찾기

우리가 갖추어야 하는 능력들을 다른 말로 표현해보자면

“애매한 상황에서 답을 찾아가는 능력”

훈련하고 계신가요? 여건이 안된다면 일단 팀장으로 승진합시다
강제로 훈련되게 됩니다...

**AI시대에 유달리
중요해지는 새로운 강점은
어떤 것들이 있을까?**

‘빠른 맥락 파악 능력’이 요구됨

- 수행을 AI가 하다보니,
- 인지부채를 따라잡으려는 맥락 파악이 별개의 명시적 일
- 판단 빈도 $\uparrow$ $\times$ 맥락 파악 비용 $\uparrow$ = AI 시대의 새 핵심 기본기.
- 어떻게 더 쉽게·빠르게 맥락을 파악할까?
- 그 작업마다 AI를 어떻게 가속에 쓸까?

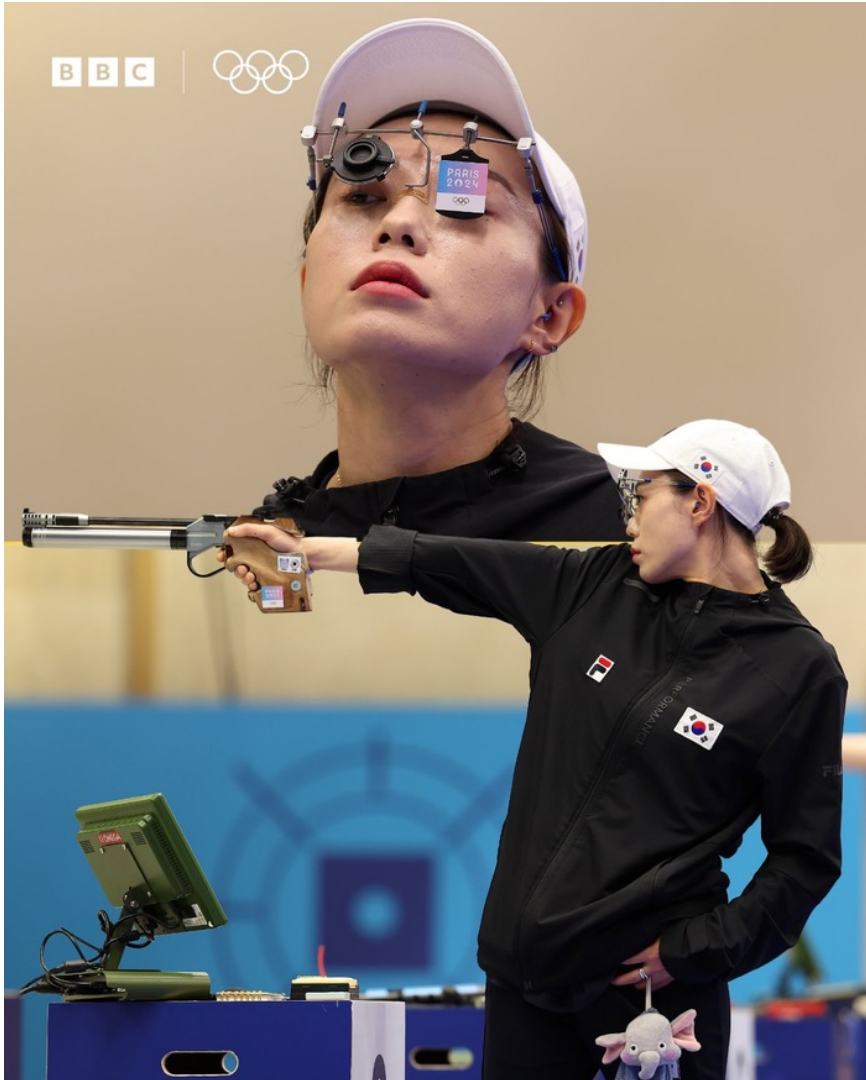
AI시대 문서에 대한 새로운 정의

- 어떤 스타트업의 '신입사원 온보딩 문서'
 - 실제 문서가 아니라, 회사 지식베이스에 LLM으로 ' 물어보아야 할 질문 리스트'
 - 신입사원은 여러 질문을 해가며, 자신에게 맞는 타입의 설명 문서를 라이브로 얻음
- 예전 세대
 - 문서란 만들어진 어떤 실체
 - 잘 정리된 문서를 찾아다녔다.
- AI native
 - 문서란 잠시 생겼다 사라지는 휘발성 가스 같은 것
 - 문서를 만들 기저 데이터(Source of Truth)만 튼튼히 있으면 된다.
- google vs chatgpt. 주로 쓰던 툴의 차이가 문서 인식에도 영향

mind-sized bites 변환력 필요

- 자신만의 능동적인 ‘생성 읽기’ 스킬이 필요
- 모두에게 적합하고 쉬운게 아니라, 특별히 나에게 맞는 방식
 - 같은 내용도 어떤 설명방식이 ‘나의 인지부하’를 적게 일으키는가
 - 자신에 대한 이해가 필요
 - 1시간짜리를 10분만에 이해되게 rewrite 하는 능력이 필요
- 내 마음이 소화할 수 있을 만큼의 한 입단위로 재조직
- 짧은 시간에 많은 context를 소화할 수 있는 능력을 기르자.

더불어 어그로력(마케팅력)이 필요하다.



- 김예지 선수
 - 파리올림픽에서 가장 유명해진 선수 (은메달)
 - 그런데 금메달도 한국 선수였다?
 - 하지만 우리는 김예지를 기억
- 서비스, 게임, IT시장도 마찬가지
 - AI로 다들 생산성 좋고, 다 잘만들면
 - 비슷한 제품이 100개가 나온다면?
- 어떻게 수백개 제품중에 간택받을 것인가?
 - 가장 중요해지는 능력
- 오히려 회사의 핵심 지표가 될 수 있다.
 - 모든 직군에게 필요한 능력이 되어가고 있음

당신에게 필요한 어그로력

- 제품 하나를 만드는 노력이 줄어들며 수천개의 서비스가 나오게 된다.
 - 모두 ai가 만드는 상황에서 탁월함을 추구하기도 쉽지 않고,
 - 동시에 그런 탁월함을 쌓았다 하더라도
 - 물량에 밀려 발견되기는 더 어려운 시대이다.
-
- 동일 수준의 제품이라면 더 어그로(마케팅)를 끄는 능력이 필요하다.
 - 이 시대는 개인이 단순히 우리가 흔히 실력이라고 말하던 요소 말고
 - 개인과 제품에 대한 막대한 스타성을 추구하는 노력 배분이 필요하다.
 - 악플보다 무플이 진정 무서운 시대이다. (그래도 악플 노노)

더불어 필요한 것은 명확한 취향(taste)

- 제품들이 너무 많다보니, 취향이 반영되지 않은 제품은
- 단순 어그로(마케팅)만으로는 지속성을 가질 수 없다.
- 다 기계가 만들어주는 AI시대이지만 오히려
- 만든 사람의 철학, 취향, 고집등이 강하게 반영되고
- 색다른 결이 묻어나오는 제품이 선택받게 된다
- 이제 우리는 전체 공정의 일부분을 맡는 전문가가 아니라
- 시작부터 끝지점까지 취향을 입히는 사람이 되어야함

AI 시대의 가장 흔한 실수

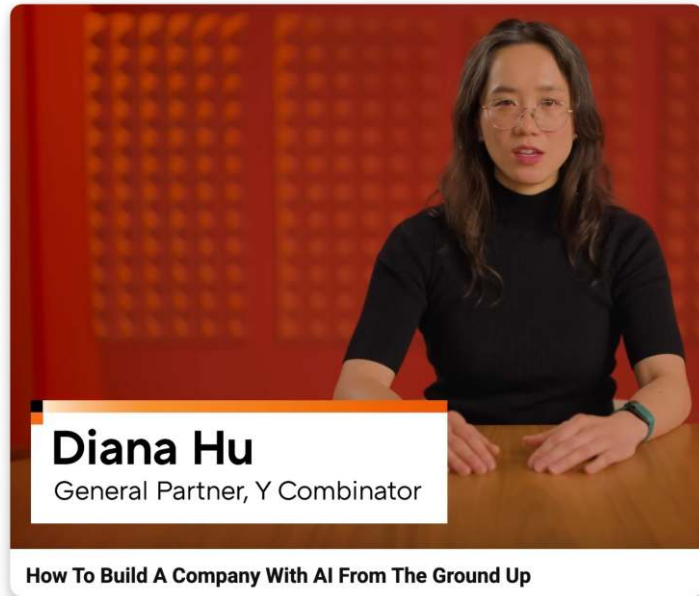
- 와~ 빠르게 만들 수 있다. → 다 만들자.
 - 향상된 개발력(?)에 취한 회사가
 - 스스로 불러온 재앙에 짓눌린다.
- 우리는 그 많은 기능들이 정말 필요했는가?
 - 아무도 쓰지 않은 기능, 쌓이는 관리포인트들
 - feature creep이 되어가며 질려가는, 또는 오지 않는 유저
- 만드는 비용이 0으로 수렴했다고 정말 다 만들어야 했는가?
 - 숟가락이 0원이라고, 우리는 100개를 쓰지 않는다.

여기서 취향이 필요하다.

- ‘무엇을 할 것인가’가 아니라 ‘무엇을 덜 할 것인가.’
- 어떤 일관성 안에서 연계되는가
- 취향은 빼기 만으로 표현되는 것은 아니지만,
- 더하는 비용이 0으로 줄어드는 시대에
- 빼는 것은 매우 명확한 취향의 발현이다.
 - 더불어 기술 부채를 피하는 길이기도 하다.
- AI가 생산해 놓는 더미들 중에서
내 이야기를 전달하는데 본질적인 것만 고르는 능력

AI native 기업으로 변신할 때 주요 병목 포인트(부제 : 취향의 반전)

앞서 말한 AI native 회사의 조건



- 자료들은 물론이고 회의와 작은 결정까지도 기록으로 남고, machine readable해서 AI가 검색하고, 참조할 수 있을 것
 - Queryable
- queryable한 회사의 정보와, 지난 시도의 결과들과 다음 시도의 입력이 되어, 더 나은 시도가 loop를 돌게 하자.
 - Closed loop
- 이를 통해 이전보다 더 나은 시도를 지속적으로 반복하게 하자
 - Self-improving

이게 되면 회사의 속도는 무한이 될까?

일단 SSOT만들기 넘 어렵다.

- Single Source of Truth
 - 회사의 모든 정보가 언제나 믿을 수 있게 유지된 싱글 소스
- AX는 커녕 다들 데이터화도 안되어 있음 (DX도 안됨)
- 작은 커뮤니케이션도 전부 디지털라이즈?
 - 사람들의 일하는 습관이 그렇게 형성 안되어 있음
- SSOT를 가지고 싶지만, 너무 많은 정보들이 파편화 되어 있고
- 한번 세팅해도 시간이 지나면서 자꾸 괴리가 일어난다.
 - 무엇이 진리인지 기계적으로 판단하기 너무 어려움
 - SSOT를 끊임없이 자동 빌드하는 체계와 시스템이 필요

사람들의 번아웃이 빨리온다.

- 어찌어찌 SSOT만들고, 그 기반으로 작업속도가 빨라지면
- 아무리 좋은 검증레이어를 만들어도
- Human in the loop를 해야할 요소는 존재하기 마련
- 이것들이 적체되면서, 결국 그 Human이 정체포인트가 됨
- 더 적은 사람수로 커버하면서, 정신적 에너지 소모
- AX시대에 오히려 멘탈, 에너지관리가 가장 중요해짐

Taste의 수렴이 어렵다.

- 이것이 진정한 문제
- AI가 무한생산 할 수 있을 때, 어떤 것을 생산할지 택하게 하는 것은 Taste 라고 했다.
- 하지만 Taste라는 것은 정답이 없고 명문화하기가 힘들다.
 - 이키텍처적 호불호, 생각하는 유저의 페르소나
 - 유저들이 처음에 겪어야 하는 UX의 흐름, 전반적인 디자인 방향성 등등등
- 하나의 제품에 여러 취향이 난립하지 않게 하기 위해 수렴 통일 해야함
- 이 과정이 구성원 인간들의 느린 커뮤니케이션(회의)에 기반. (인원수 비례 속도 저하)
- AI의 생산의 속도보다 인간의 taste의 수렴속도가 병목이 된다.
- taste가 수렴되지 않아 연결된 후속 모든 작업들이 일시정지 하게 된다.
 - 이 중간적인 sync 프로세스가 다시 AI 회사의 속도를 일반 회사 속도로 떨어뜨리는 것을 느낀다.

데이터 직군의 미래는?

데이터 직군은 읍니다.

- 가장 많이 채용이 줄어든 직군중에 하나 '데이터'
 - 왜 그런가?
- 왜 amplitude? 데이터분석?
- 원본 데이터 소스에 claude와 mcp연결해두면
- AI가 소스 접근과 분석과 대시보드까지 다 뽑아주는데
- 여전히 분석가 필요한가요?

회사의 AI-native 화에 길이 있다.

 Combinator

Build a queryable company

- **Make the org legible to AI**
Record everything, minimize DMs/emails
- **Every action creates an artifact than can self improve**
Embed agents and codify manual “glue”
- **One shot internal dashboards for every company ops**
Revenue, sales, eng, hiring



AX를 위한 DX는 누가 잘할까?

- 내가 데이터 일 할 때 가장 많이 했던 말
 - 모든 회사의 데이터는 2가지 타입이다
 - 없거나, 쓸 수 없거나
- AX시대에도 마찬가지
 - 대부분의 회사가 AX이전에 필수인 DX조차 잘 안되었다.
- 기존 DX의 가장 큰 문제가 뭐였나? 목적없는 DX
- AX라는 목적에 기반한 DX를 가장 잘할 수 있는 사람
 - 현실과 가장 괴리가 적은 SSOT Dataset 을 만들 수 있는 사람
 - 데이터 분석, 데이터 엔지니어, 데이터 사이언스 직군
- AX 책임자로 승진해서 이직하자. SSOT 그거 내가 할 수 있다!

**가장 궁금한 질문.
그럼에도 이 시대에
전문성은 여전히 필요한가?**

겔만 암네시아 이펙트

Gell-Mann Amnesia Effect

- “신문에서 내 전문 분야 기사를 읽으면 오류투성이가 보이는데,”
- “다른 비전문 분야 기사를 읽으면 그건 정확할 거라고 믿는 망각 현상.”
- 신문 → LLM으로 매체만 바뀐 정확히 같은 편향이 존재
- **AI의 결과들이 그럴싸해보였던 것 중 일부는 우리가 비전문가였기 때문**
 - 체계적 인지 편향일 가능성이 있다.
- ‘그럴듯한 가짜’를 걸러내고 ‘진짜’를 확신하려면, 전문성이 필요하다.
- **즉 가장 타당하고 안전한 검증 레이어를 만들 수 있는 건 그 분야의 전문가**

쉬운 결정은 AI가 다 하고나면 사람에게 올라오는 것은 **어려운 결정**뿐

- AI가 못하고 올리는 것은 보통 가치를 다루는 어려운 트레이드 오프 질문
 - ‘옳은 가치1’ vs ‘옳은 가치2’
- 효율성
 - 전문가가 10분만에 쟁점을 파악하고, 비전문가는 1시간이 걸려 파악했다.
 - 결정에 걸리는 시간은 모든 것이 멈춘다. 누구를 써야하나?
- 정확성
 - 비전문가는 ‘부존재’를 눈치못챈. (있어야 할 게 없는 데, 눈치 채셨나요?)
 - AI가 미처 검토하지 못한 각도의 발견이 정확성을 가져온다.

책임이 부가되는 결정은 인간이.

- 완전 자율 AI 식당이라도, 길 가는 6살에게 책임을 맡길 수 없다.
 - 식중독 나면?
- 세상의 많은 결정들 중 어려운 결정들은 '책임이 딸린 결정'
 - 실패 시 어떤 문제가 생길 지 이해하고, 예측할 수 있고
 - 실제 그 문제를 수습할 수 있는 주체가 결정을 내려야 한다.
 - 책임의 자격에는 타인의 납득이 필요하다.
 - 현재로서는 AI는 책임을 질 수 있는 주체가 아니다.
- 사회적으로 용인되는, 도장을 찍을 수 있는 적격자가 여전히 필요

긴 시간 함께 살펴보았다.

오늘의 이야기를 돌아봅시다.

- 회사와 개인이 AX를 하면서 겪게 되는 **시련의 스텝들**
- AI가 만든 산출물이 다음 발전을 방해하는 **기술부채**
- 생성된 결과를 읽지도 않고 딸깍 내보내며 쌓이는 **인지부채**
- 돌아는 가지만, 왜 이런 세팅인지 다시는 알 수 없는 **의도부채**
- 인간의 일은 이제 **생산에서 검증으로** 넘어가는 시대
- 두터운 검증 안에서 **24시간 일하는 AI**들을 키우는 세상
- **암묵지를 자동 누적**하는 체계가 있어야 반복이 줄고 발전이 있다.
- 이 시대의 인재상은 '**애매한 상황에서 답을 찾아가는 능력**'을 가진 자
- 다 AI가 할 것 같았지만, **가장 어려운 선택은 결국 전문가의 몫**

전문가 = 스킬의 숙련자에서 운영의 책임자로

- = 자신의 도메인의 SI를 만들고 운영하는 사람
- = 풀고 싶은 문제와 바라는 결과를 세심하게 정의
- = 만드는 단계의 검증 레이어를 만들고
- = 운영하는 단계의 검증 레이어를 유지보수 한다.
- = 각 단계에서 선택을 위해 도메인 taste가 필요
- = 책임이 따르는 어려운 결정을 내리고
- = 운영에 따른 결과를 책임지는 사람

옳은 가치 판단
지지 받는 취향
납득 되는 책임

AI시대에도 여전히 사람의 일은 남아 있습니다

**살아남는 걸 넘어
저 멀리 가봅시다**



감사합니다.

2026.6.11

하용호

yongho.ha@dataoven.ai